

7 组合逻辑电路与器件

7.1 组合逻辑电路基本概念和器件符号

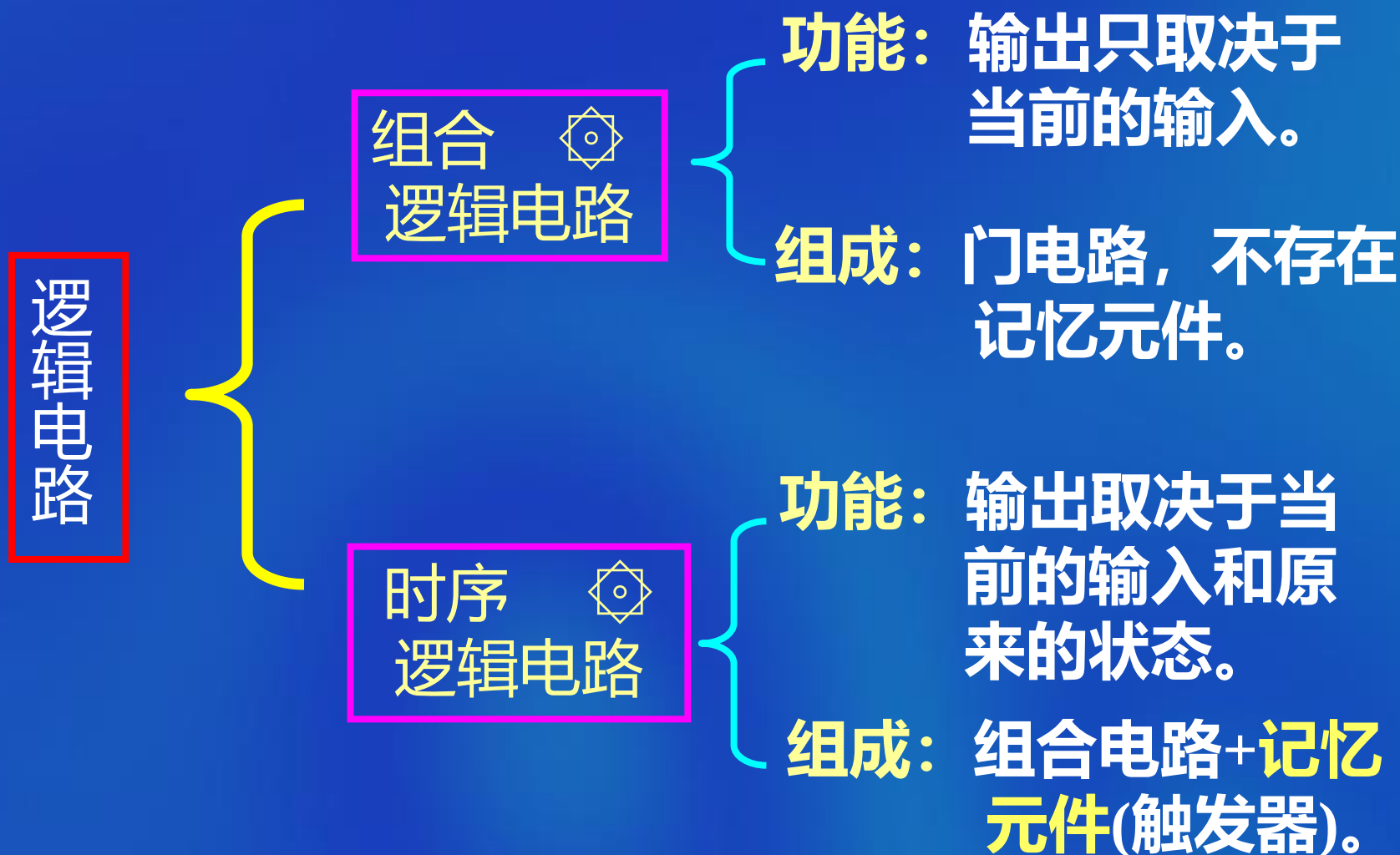
7.2 译码器与编码器

7.3 多路选择器与多路分配器

7.4 加法器和比较器

7.5 算术/逻辑运算单元(ALU)

7.1 组合逻辑电路基本概念和器件符号



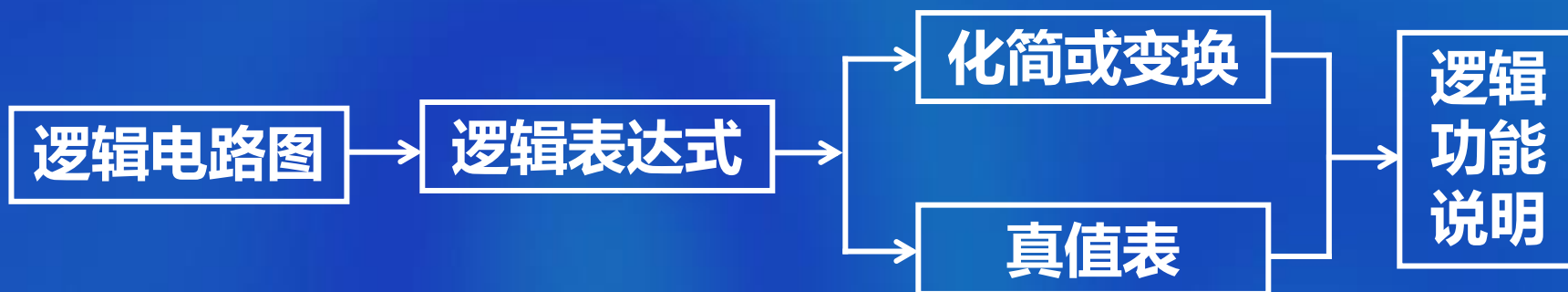
组合逻辑电路中的两类问题



门级组合逻辑电路的基本单元是各种基本门电路。

分析方法

组合逻辑电路分析流程图



[例] 分析图示电路的逻辑功能。

[解] (1) 写出逻辑表达式

$$L_1 = \overline{CB}$$

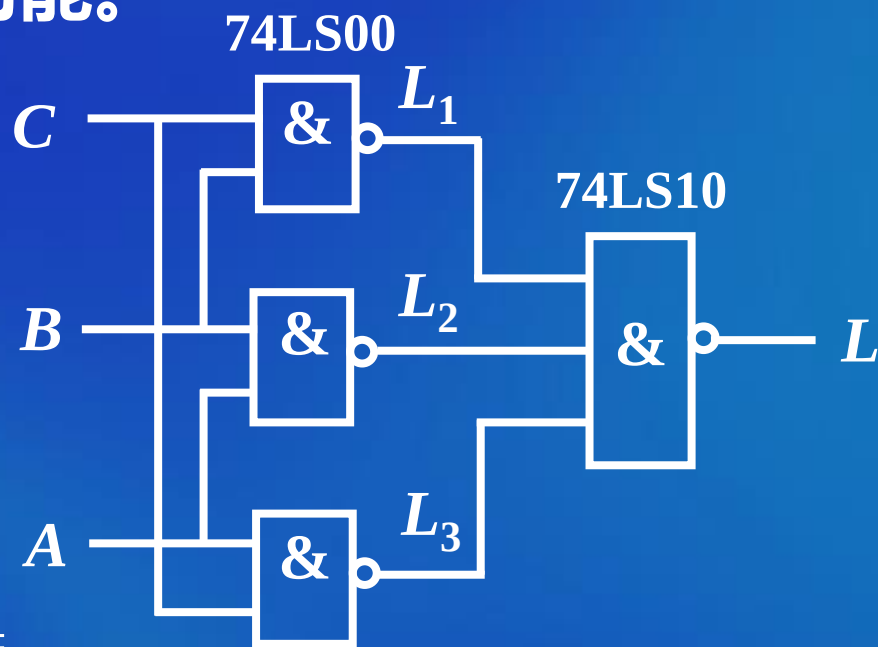
$$L_2 = \overline{AB}$$

$$L_3 = \overline{AC}$$

$$L = \overline{L_1 \cdot L_2 \cdot L_3} = \overline{\overline{CB} \cdot \overline{BA} \cdot \overline{AC}}$$

(2) 变换逻辑函数

$$L = \overline{\overline{CB} \cdot \overline{BA} \cdot \overline{AC}} = CB + BA + AC$$



$$L = CB + BA + AC$$

(3) 根据函数表达式列真值表

(4) 分析逻辑功能

输入变量A、B和C中有两个以上取值为1时，输出函数
 $L=1$ ；否则 $L=0$ 。

输入	输出
ABC	L
0 0 0	0
0 0 1	0
0 1 0	0
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	1
1 1 1	1

三变量的多数表决器。

设计方法

门级组合逻辑电路设计方法流程图

根据逻辑命题选取输入和输出逻辑变量。

用二值逻辑的0和1分别代表输入和输出逻辑变量的两种不同状态，称为逻辑赋值。

由真值表写出逻辑函数表达式。根据选用逻辑门的类型，将函数式化简或变换为最简式。



[例] 为燃油蒸汽锅炉设计一个过热报警装置。用三个数字传感器分别监视燃油喷嘴的开关状态、锅炉中的水温 and 压力是否超标。当喷嘴打开且压力或水温过高时，都应发出报警信号。

[解] (1) 列真值表

A表示压力逻辑状态: 1过高,0正常。

B表示锅炉水温逻辑状态: 1过高,0正常。

C表示喷嘴开关逻辑状态: 1开, 0关。

L表示报警信号: 1报警, 0正常。

输入	输出
<i>CBA</i>	<i>L</i>
0 0 0	0
0 0 1	0
0 1 0	0
0 1 1	0
1 0 0	0
1 0 1	1
1 1 0	1
1 1 1	1

(2) 写出逻辑函数表达式

$$L = \overline{C}BA + C\overline{B}\overline{A} + CBA$$

(3) 化简逻辑函数为最简与或式

$$L = CA + CB$$

将上式变换为与非 - 与非表达式

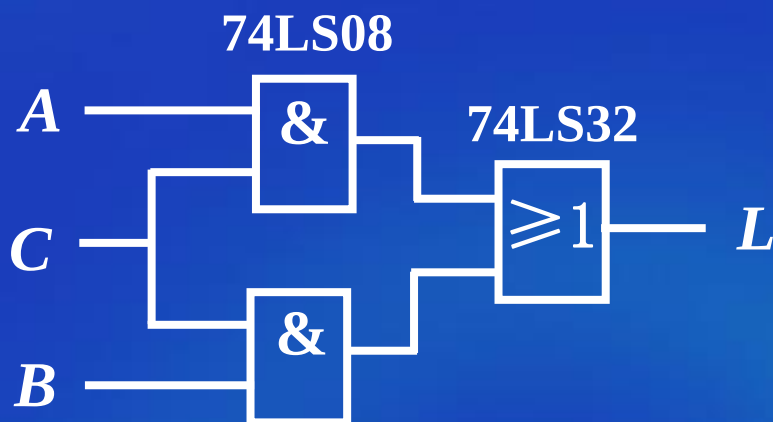
$$L = CA + CB = \overline{\overline{CA} \cdot \overline{CB}}$$

真值表

输入	输出
<i>CBA</i>	<i>L</i>
0 0 0	0
0 0 1	0
0 1 0	0
0 1 1	0
1 0 0	0
1 0 1	1
1 1 0	1
1 1 1	1

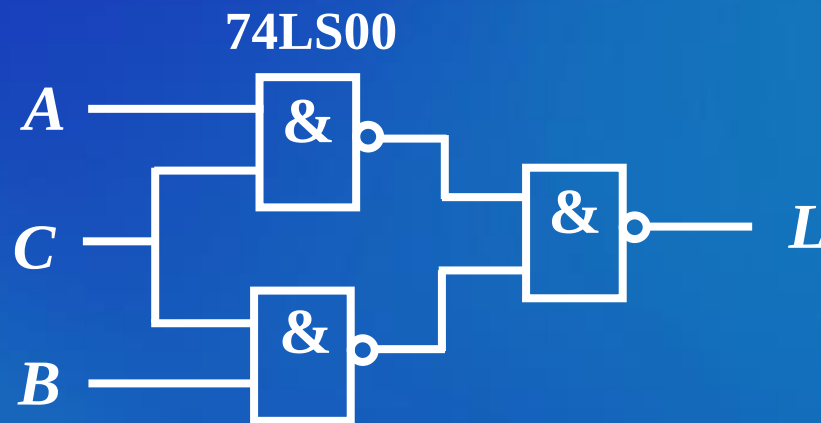
(4) 画逻辑电路图

(a) 用与门和或门实现



$$L = CA + CB$$

(b) 用与非门实现



$$L = \overline{\overline{CA}} \cdot \overline{\overline{CB}}$$

若用集成门实现与或式，至少需要两种类型的门电路。

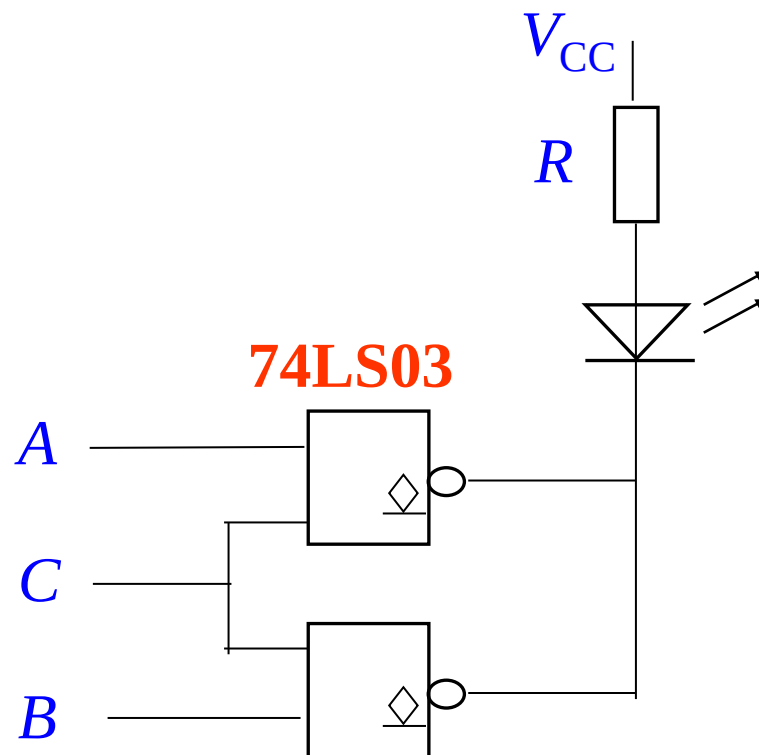
若用集成门实现与非式，则仅需要一种类型的门电路。

如果要求用OC门驱动，用发光二极管显示报警？

$L = CB + CA$ 灯亮

$$\overline{L} = \overline{CB} \cdot \overline{CA}$$

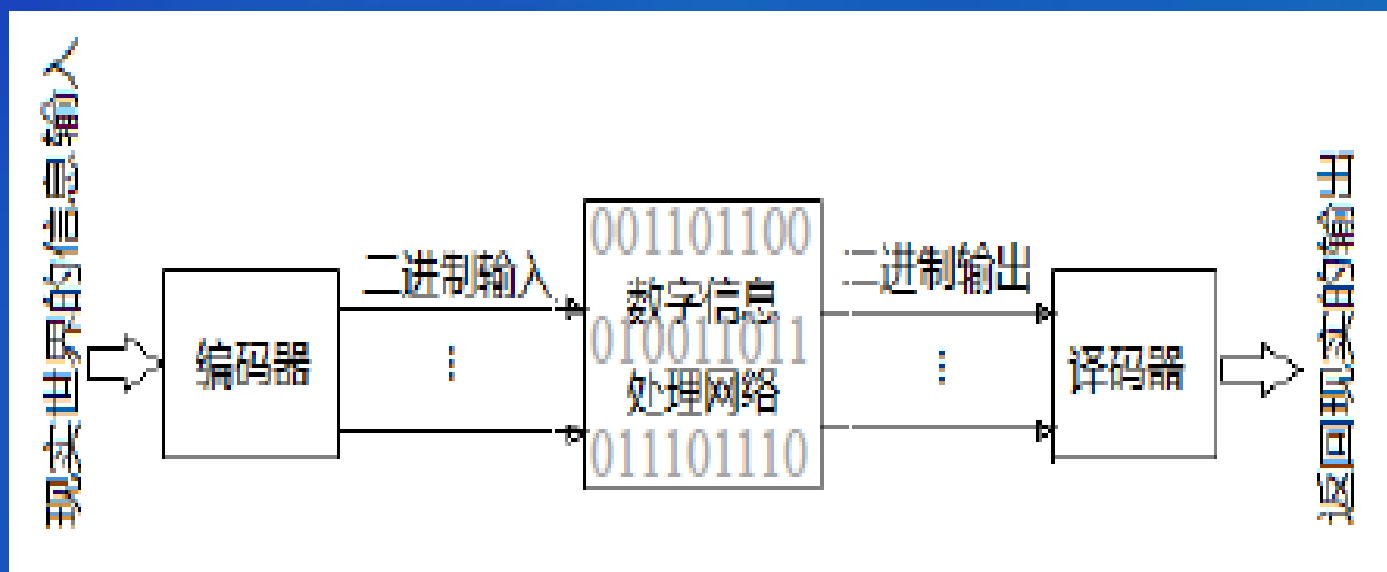
要接上拉电阻；
低电平驱动能力强；
可线与



3.2 译码器与编码器

编码器 (广义) : 将现实世界的信息转换为数字信息处理网络可以理解的二进制信息;

译码器 (广义) : 将数字网络处理的结果转换为现实世界可接受的信号输出。

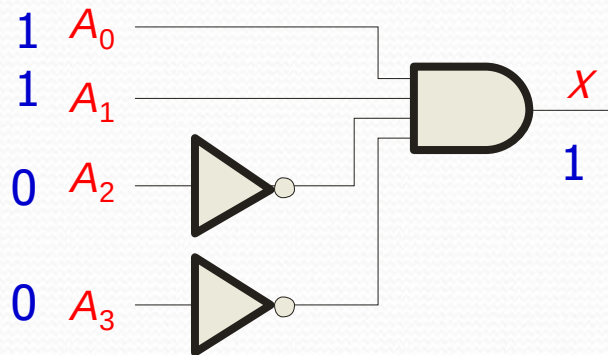


The basic binary decoder

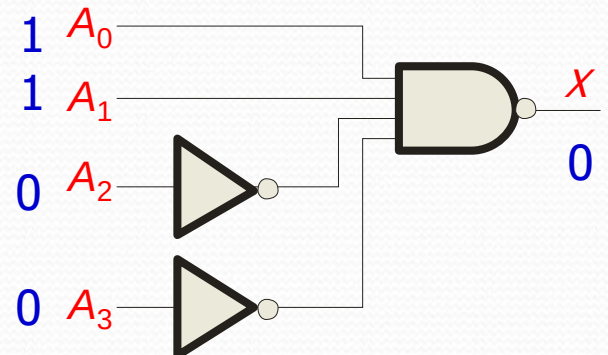
Suppose that we need to determine when a binary 0011 occurs on the inputs of a digital circuit.

An AND gate and inverters are used to produce a HIGH output only when inputs are 0011. It is **the active-HIGH output decoder** (高有效输出的译码器) .

If a NAND is used in place of (代替) the AND gate, the output is a LOW only when inputs are 0011. It is **the active-LOW output decoder** (低有效输出的译码器) .



Active HIGH decoder for 0011



Active LOW decoder for 0011

具有译码功能的电路称为译码电路，而相应的MSI芯片称为译码器（Decoder）。

这种译码器被称为
 $n-2^n$ 线译码器。

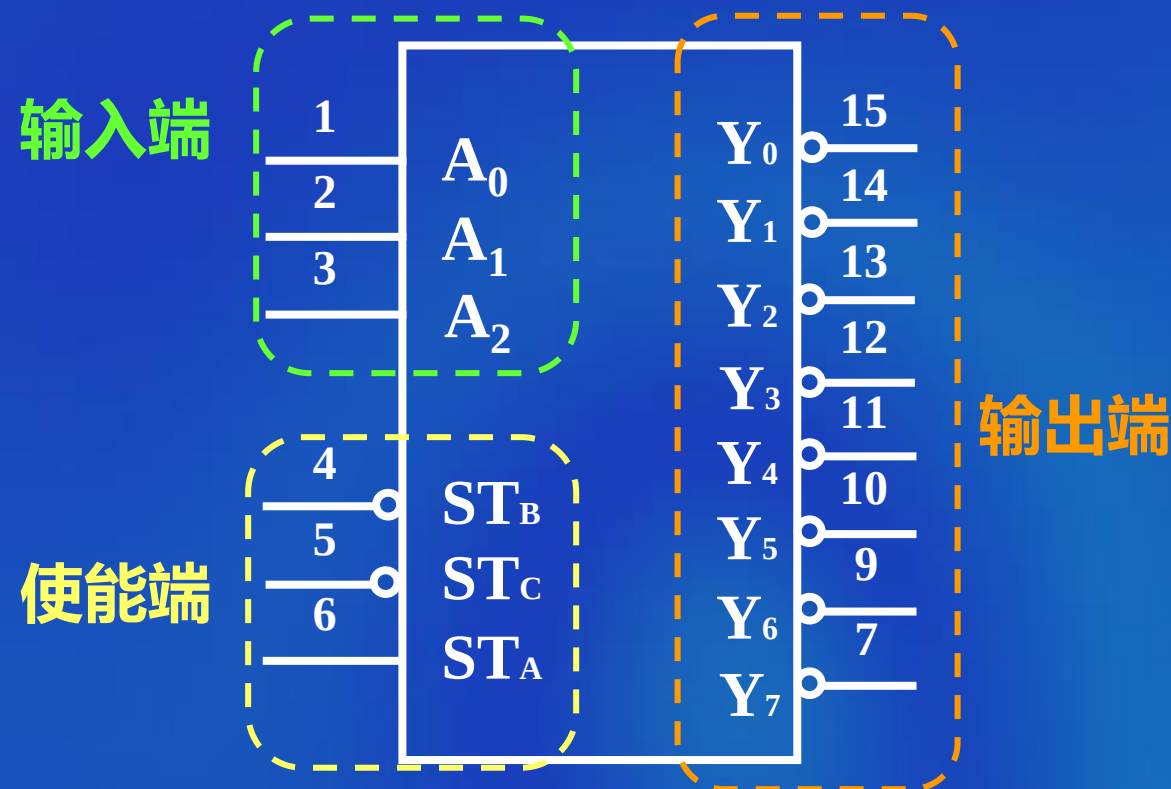
译码输入： n 位二进制代码

译码输出： m 个输出信号 $m=2^n$

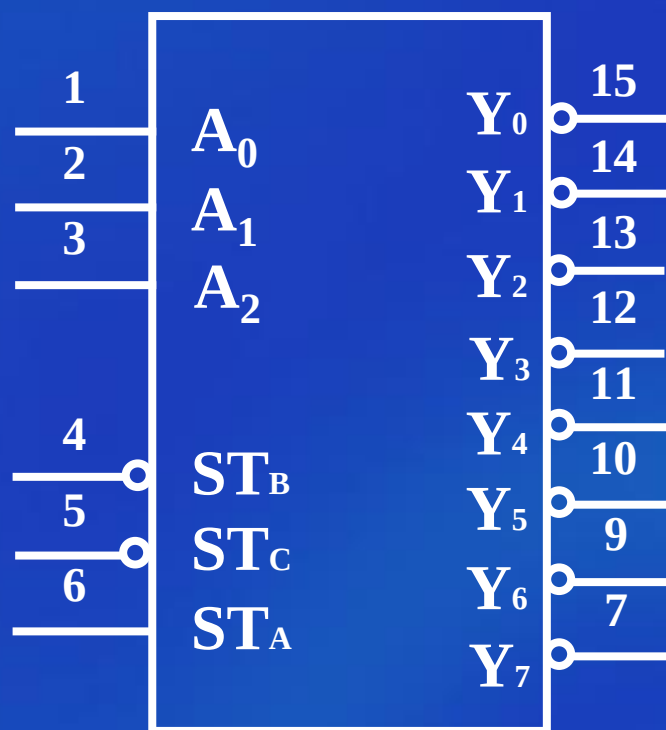
译码规则：对应输入的一组二进制代码有且仅有一个输出端为有效电平，其余输出端为相反电平

1. 基本译码器的功能描述(以74LS138为例)

74LS138符号图



74LS138是最常用的集成译码器之一，它有三个译码输入端 A_2 、 A_1 和 A_0 ，八个输出端，因此又称为3-8译码器。



ST_A 、 ST_B 和 ST_C 是三个控制输入端(使能控制端)。

当 $ST_A=1$, $\overline{ST_B} + \overline{ST_C} = 0$ 时, 译码器处于工作状态。

当 $ST_A=0$ 或者 $\overline{ST_B} + \overline{ST_C} = 1$ 时, 译码器被禁止(即译码器不工作)。

使能端的用途: 扩展输入变量数(功能扩展)。

功能表

控制输入		译码输入			输 出							
ST_A	$\overline{ST_B} + \overline{ST_C}$	A2	A1	A0	$\overline{Y_0}$	$\overline{Y_1}$	$\overline{Y_2}$	$\overline{Y_3}$	$\overline{Y_4}$	$\overline{Y_5}$	$\overline{Y_6}$	$\overline{Y_7}$
×	1	×	×	×	1	1	1	1	1	1	1	1
0	×	×	×	×	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1	1	1
1	0	1	0	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0

控制输入		译码输入			输 出							
ST_A	$\overline{ST_B} + \overline{ST_C}$	A2	A1	A0	$\overline{Y_0}$	$\overline{Y_1}$	$\overline{Y_2}$	$\overline{Y_3}$	$\overline{Y_4}$	$\overline{Y_5}$	$\overline{Y_6}$	$\overline{Y_7}$
×	1	×	×	×	1	1	1	1	1	1	1	1
0	×	×	×	×	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1	1	1
1	0	1	0	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0

译码器输出端的逻辑函数式为

$$\overline{Y_0} = \overline{A_2} \overline{A_1} \overline{A_0} = \overline{m_0}$$

$$\overline{Y_1} = \overline{A_2} \overline{A_1} A_0 = \overline{m_1}$$

$$\overline{Y_2} = \overline{A_2} A_1 \overline{A_0} = \overline{m_2}$$

$$\overline{Y_3} = \overline{A_2} A_1 A_0 = \overline{m_3}$$

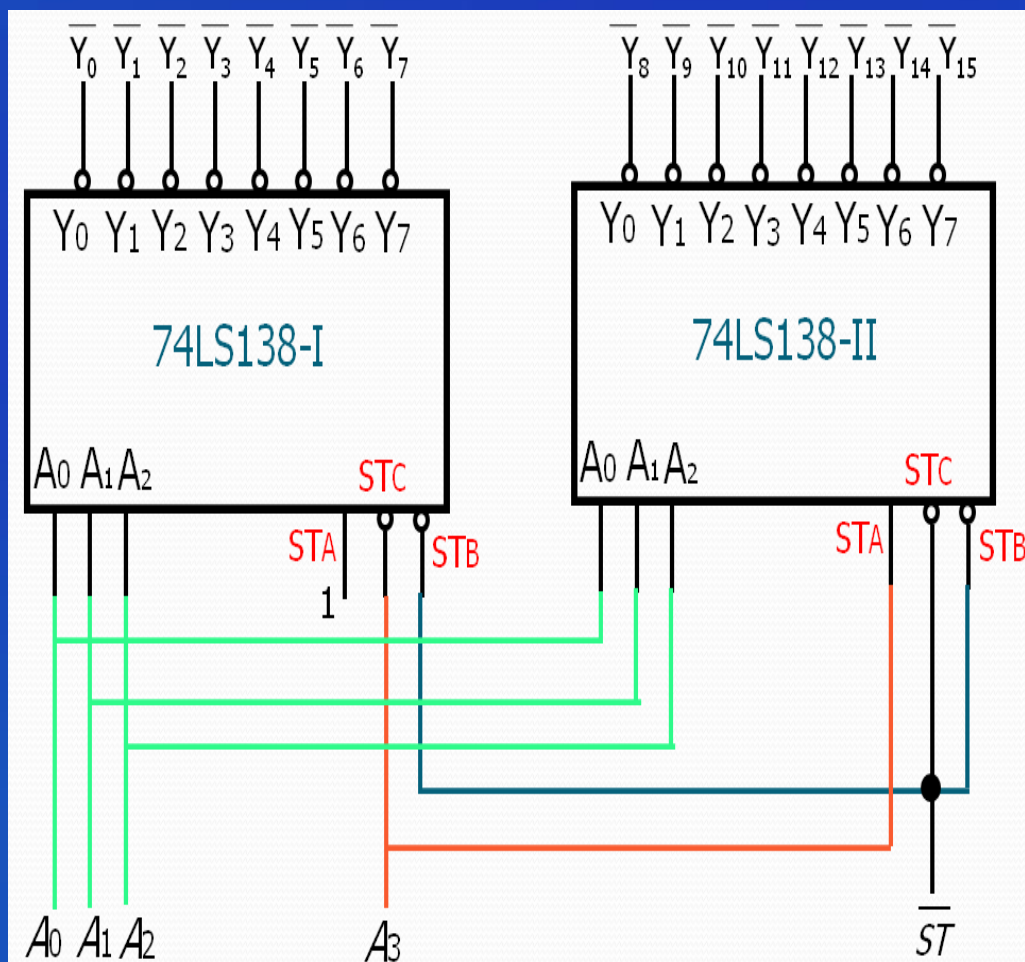
$$\overline{Y_4} = A_2 \overline{A_1} \overline{A_0} = \overline{m_4}$$

$$\overline{Y_5} = A_2 \overline{A_1} A_0 = \overline{m_5}$$

$$\overline{Y_6} = A_2 A_1 \overline{A_0} = \overline{m_6}$$

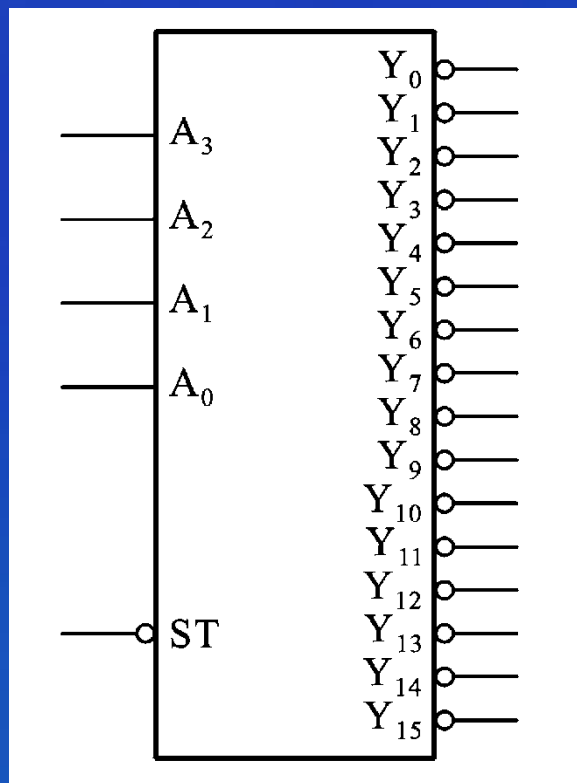
$$\overline{Y_7} = A_2 A_1 A_0 = \overline{m_7}$$

译码器的扩展(3-8译码器扩展为4-16译码器)



	A3A2A1A0	output
Chip 1 enabled	0000	$\overline{Y}_0=0$
	0001	$\overline{Y}_1=0$
	0010	$\overline{Y}_2=0$
	0011	$\overline{Y}_3=0$
	0100	Y_4
	0101	Y_5
	0110	Y_6
	0111	Y_7
Chip 2 enabled	1000	Y_8
	1001	Y_9
	1010	Y_{10}
	1011	Y_{11}
	1100	Y_{12}
	1101	Y_{13}
	1110	Y_{14}
	1111	Y_{15}

4-16译码器功能框图



常用的4 - 16线MSI译码器：
74LS154、CD4514B、
CD4515B等。

译码器的应用

(1) 实现逻辑函数

由于译码器的每个输出端对应一个输入变量的最小项，而任何逻辑函数都可写为最小项之和的表达式，因此可用这类译码器方便地构成多输出的逻辑函数发生器。

[例1] 试用3—8译码器74LS138实现函数：

$$F_1 = \sum m(0,4,7)$$

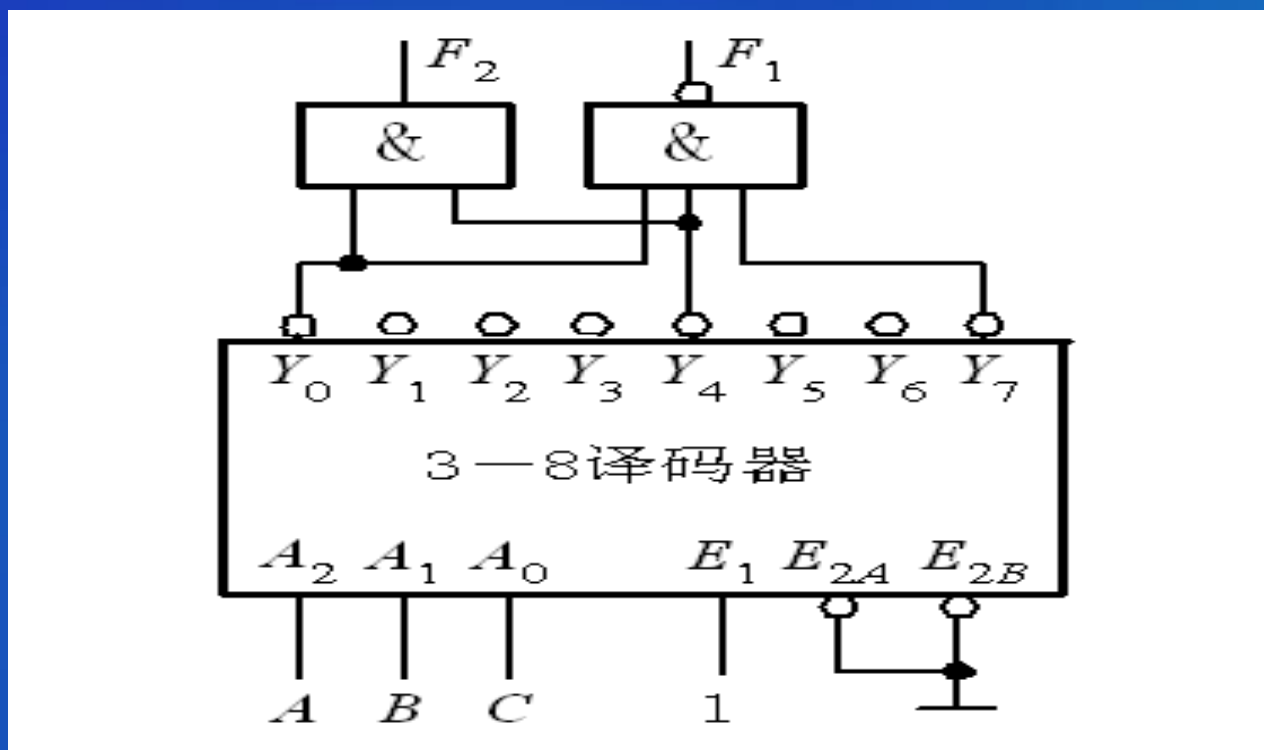
$$F_2 = \sum m(1,2,3,5,6,7)$$

[解] 因为 F_1 、 F_2 均为三变量函数，首先令函数的输入变量 $ABC=A_2A_1A_0$ ，然后将 F_1 、 F_2 变换为译码器输出的形式：

$$F_1 = \sum m(0,4,7) = m_0 + m_4 + m_7 = \overline{\overline{\overline{m_0 m_4 m_7}}}$$

$$F_2 = \sum m(1,2,3,5,6,7) = \overline{\overline{\overline{\overline{m_1 m_2 m_3 m_5 m_6 m_7}}}}$$

$$\overline{F_2} = \sum m(0,4) = \overline{\overline{m_0 m_4}} \Rightarrow F_2 = \overline{m_0 m_4}$$



[例2] 试用3—8译码器74LS138实现多输出函数:

$$L_1 = A \oplus B \oplus C$$

$$L_2 = (A \oplus B)C + AB$$

[解]

$$L_1 = \overline{C}BA + \overline{C}\overline{B}\overline{A} + C\overline{B}\overline{A} + CBA$$

$$= m_1 + m_2 + m_4 + m_7$$

$$L_2 = \overline{C}BA + C\overline{B}\overline{A} + C\overline{B}A + CBA$$

$$= m_3 + m_5 + m_6 + m_7$$

(2) 根据逻辑函数的变量个数确定译码器规模。

选用3-8线译码器74LS138。

$$\text{令 } CBA = A_2 A_1 A_0$$

将 L_1 、 L_2 写为如下形式

$$L_1 = \overline{\overline{m_1} \cdot \overline{m_2} \cdot \overline{m_4} \cdot \overline{m_7}} = \overline{\overline{Y_1} \overline{Y_2} \overline{Y_4} \overline{Y_7}}$$

$$L_2 = \overline{\overline{m_3} \cdot \overline{m_5} \cdot \overline{m_6} \cdot \overline{m_7}} = \overline{\overline{Y_3} \overline{Y_5} \overline{Y_6} \overline{Y_7}}$$

(3) 画电路



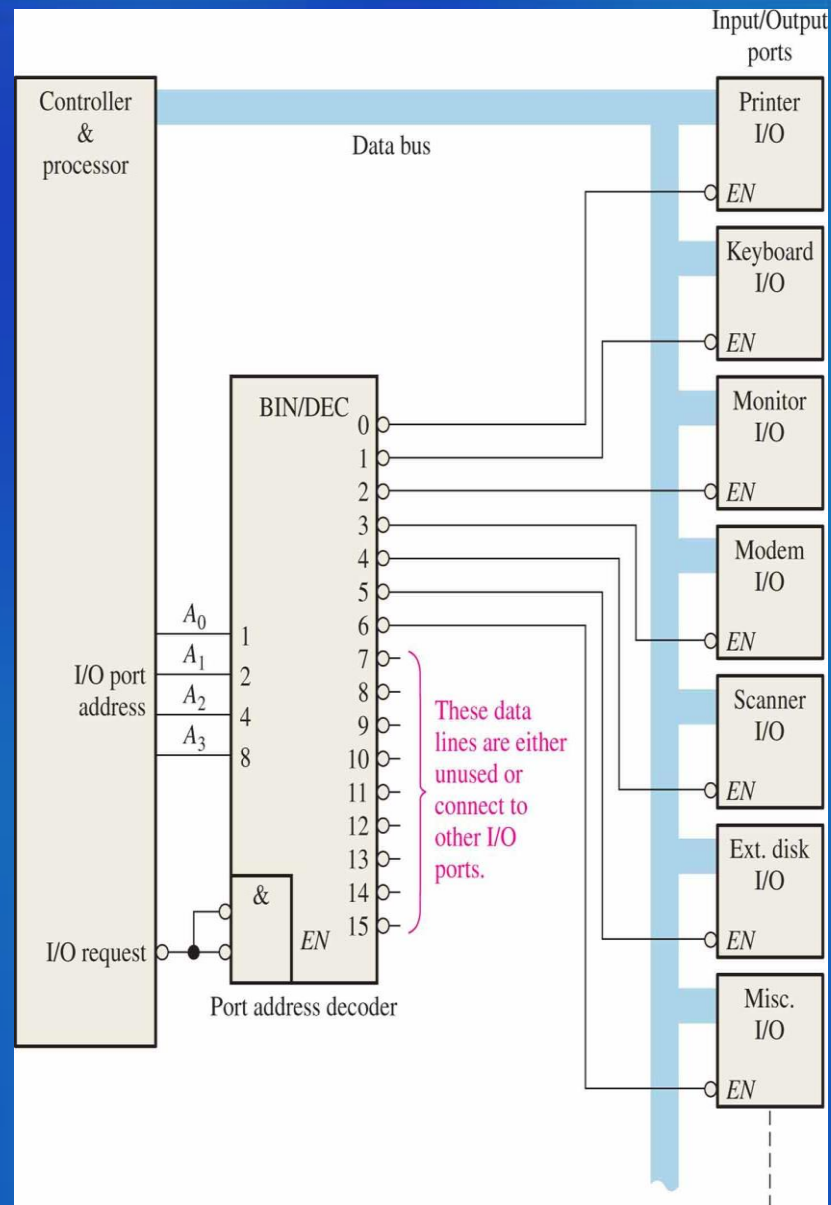
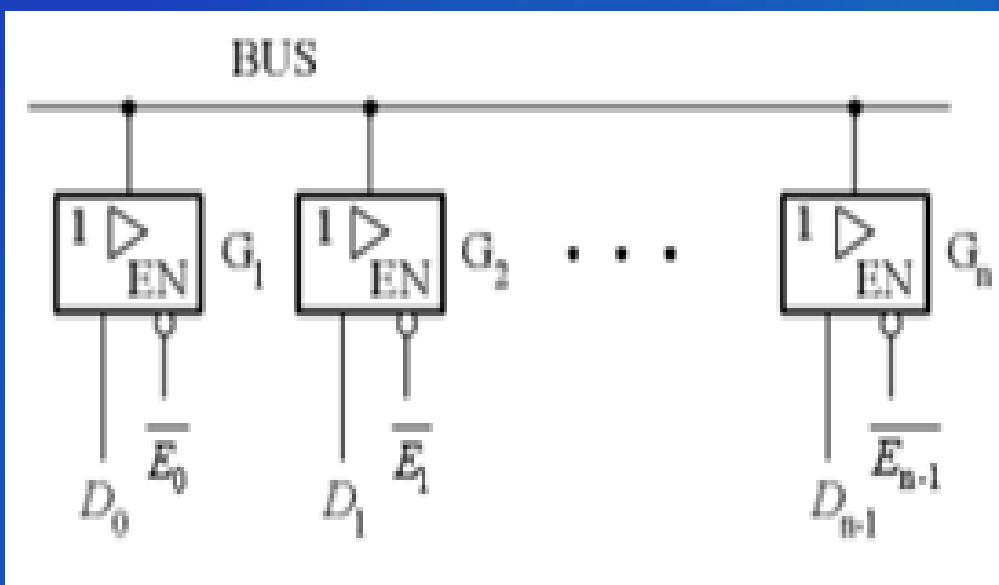
注意事项:

- 要标注输入输出变量。确定最小项编号时的变量高低顺序要与译码器输入高低顺序对应。
- 所有控制端要直接全部使能。

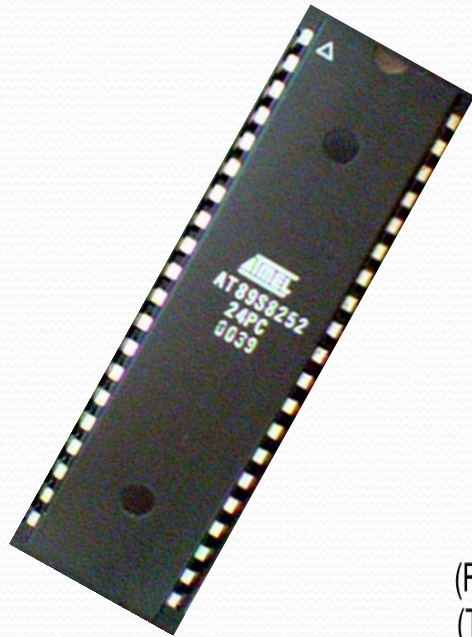
(2) 实现端口地址译码

每个外设选通时对应的译码输入称为该外设的地址码。每个外设有一个唯一的地址码来识别它。

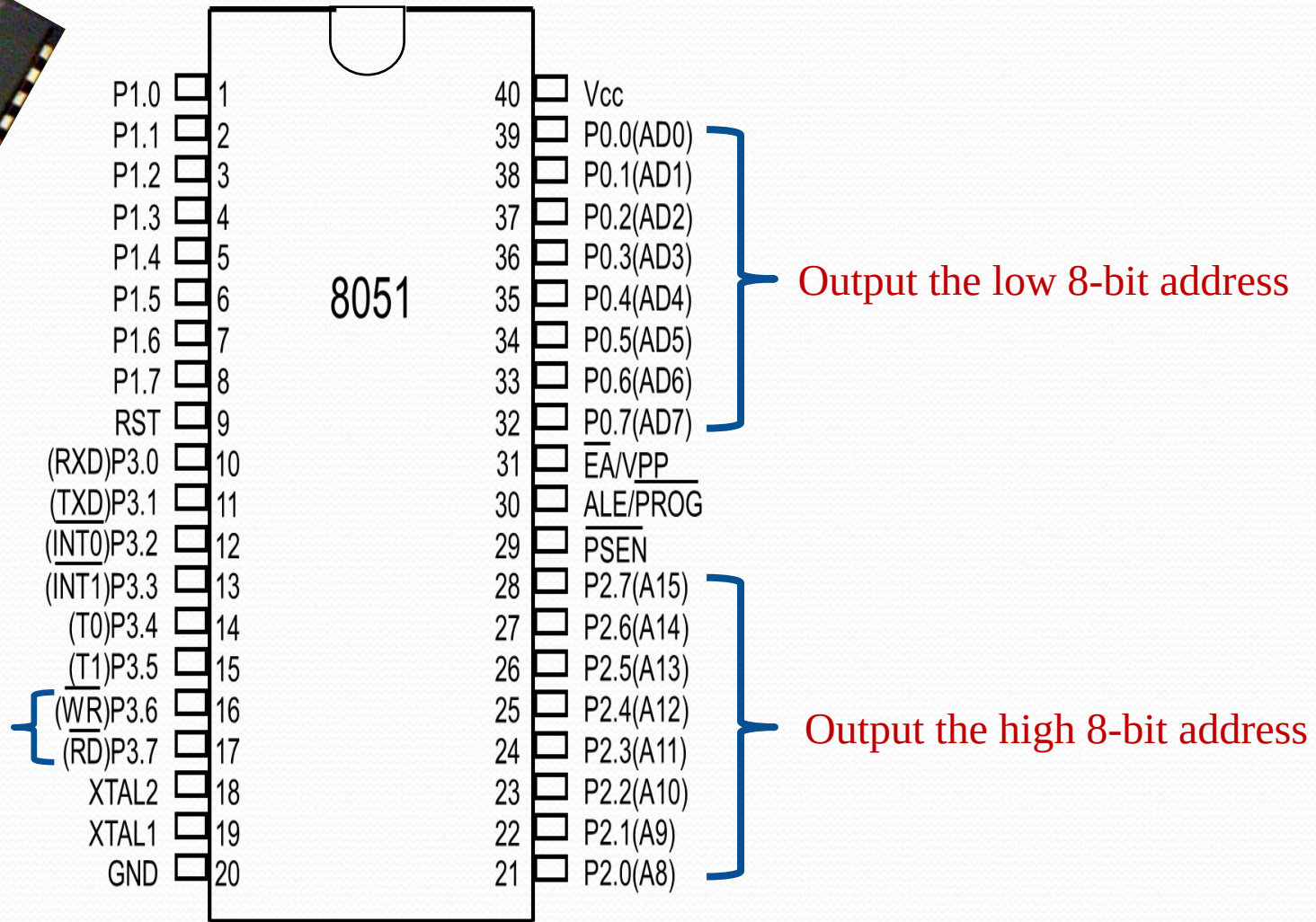
问题：如何提供 $\overline{EN}$ 使能信号？



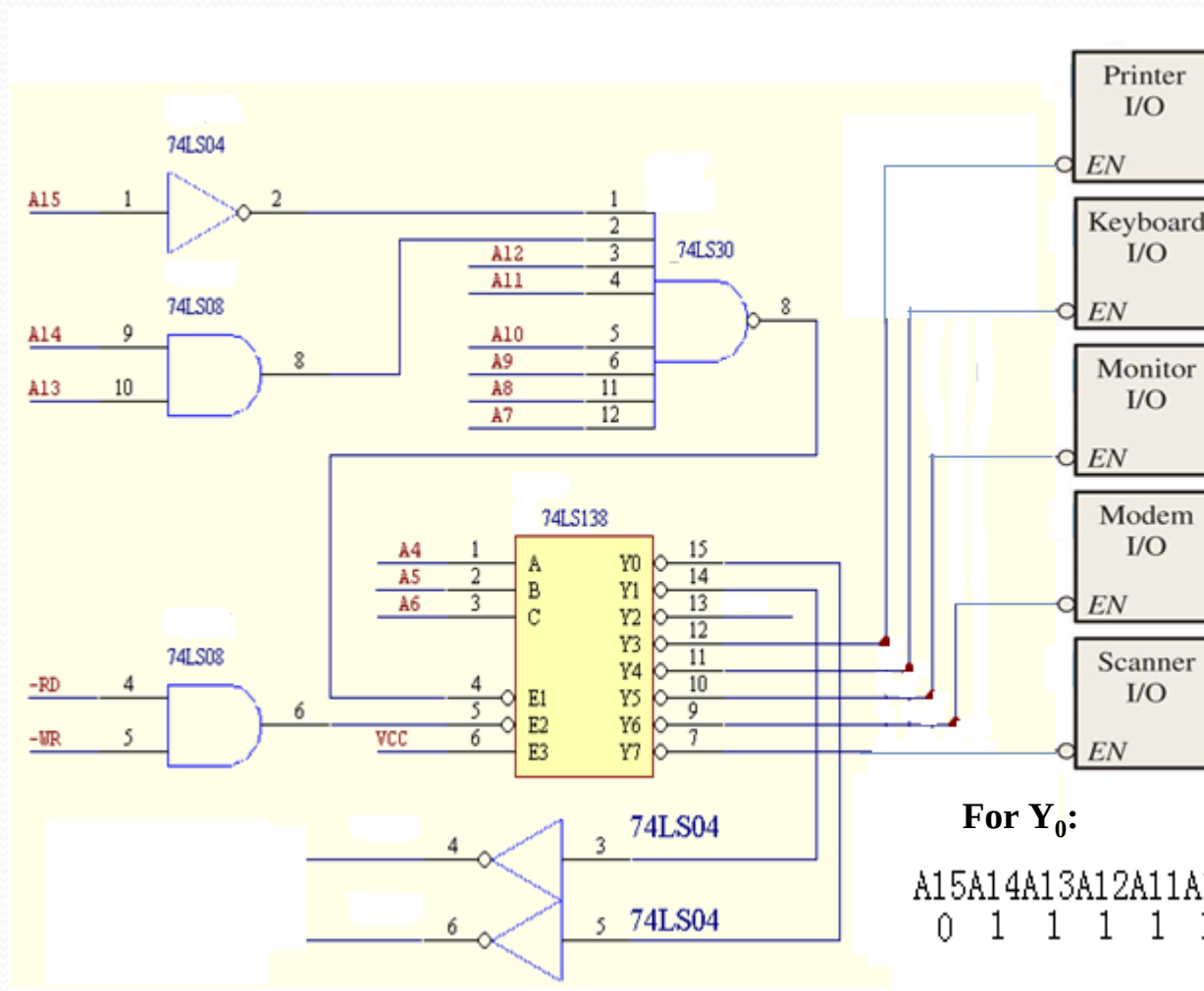
8051 Microcontroller



Output a low logic level when writing the data to or reading the data from external devices.



一个微控制器中的端口地址译码电路



Question: determine the port address of the connected 5 devices.

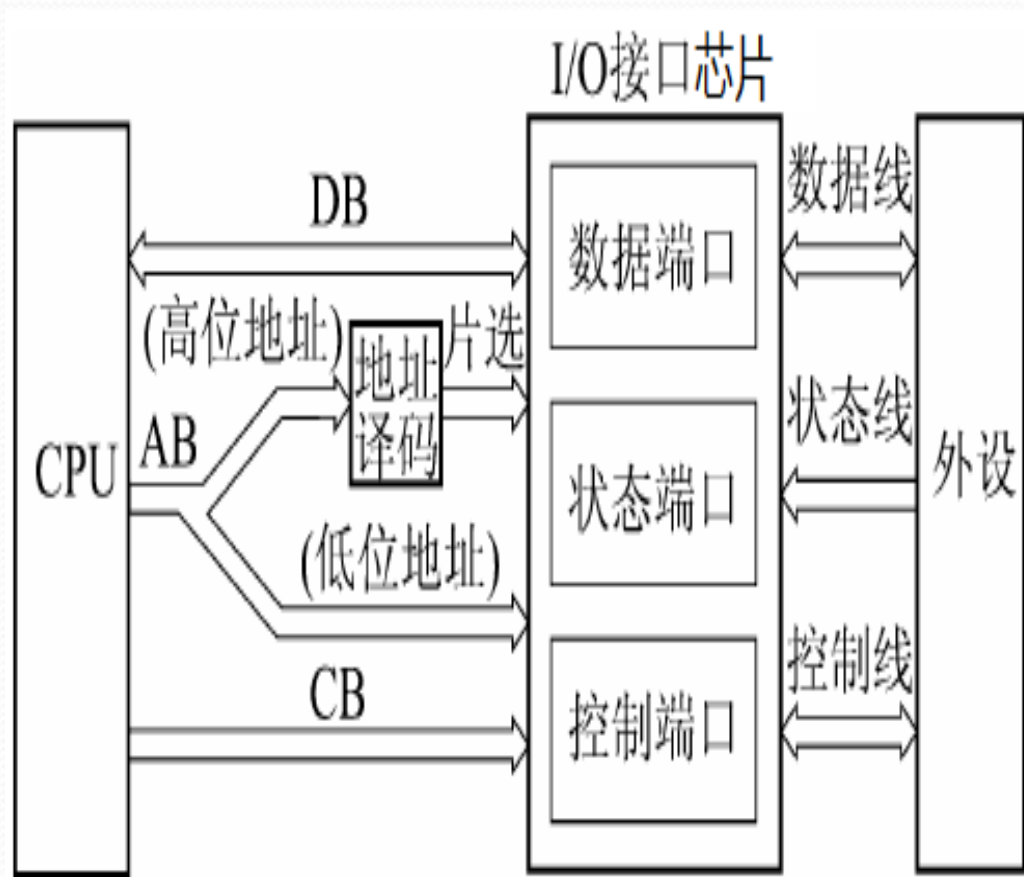
7FB0H-7FBFH,
7FC0H-7FCFH,
7FD0H-7FDFH,
7FE0H-7FEFH,
7FF0H-7FFFH.

Partial decoding

For Y_0 :

A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	
0	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	= 7F80 H
																?
0	1	1	1	1	1	1	1	1	0	0	0	1	1	1	1	= 7F8F H

I/O接口

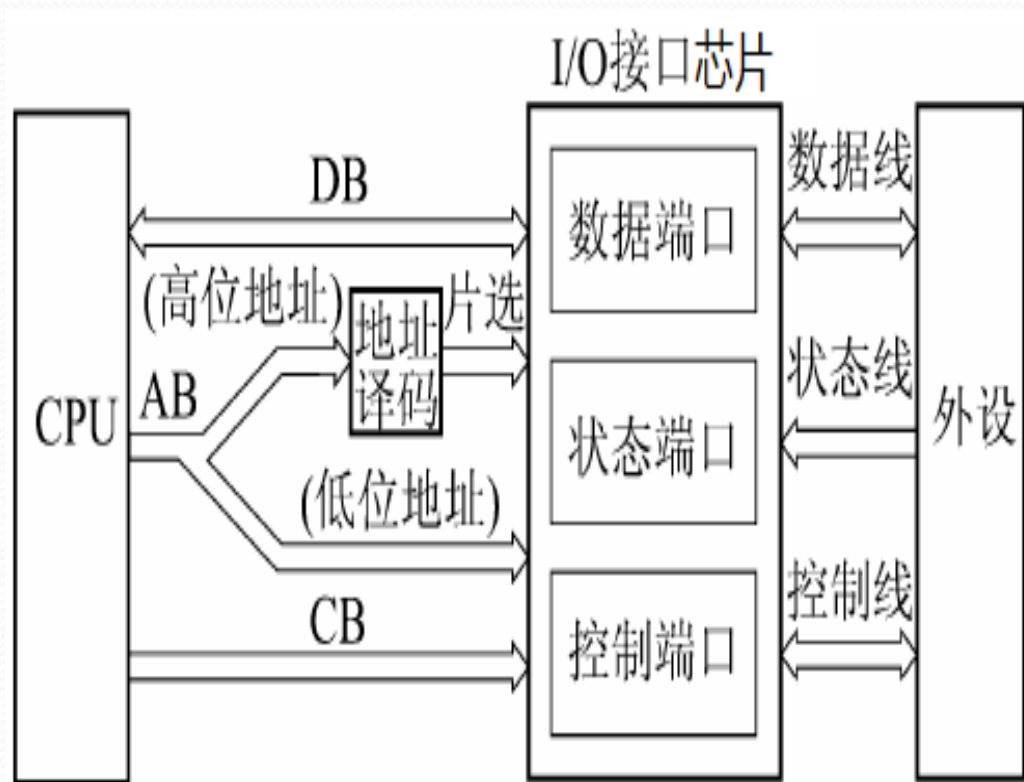


在实际应用中，有些微处理器系统，如计算机，其外设和CPU间往往有I/O接口芯片作为数据交换的中转站。

I/O接口除完成CPU和外设间信号逻辑电平的匹配外，还可完成速度和时序的匹配、驱动能力的协调等。

CPU与I/O接口交换的信息：
数据信息（要传送的数据）、
状态信息（设备当前状态，如输入设备是否准备好，输出设备是否处于忙）
和**控制信息**（读写命令），这三种信息都是以数据形式存放在接口电路的寄存器中。

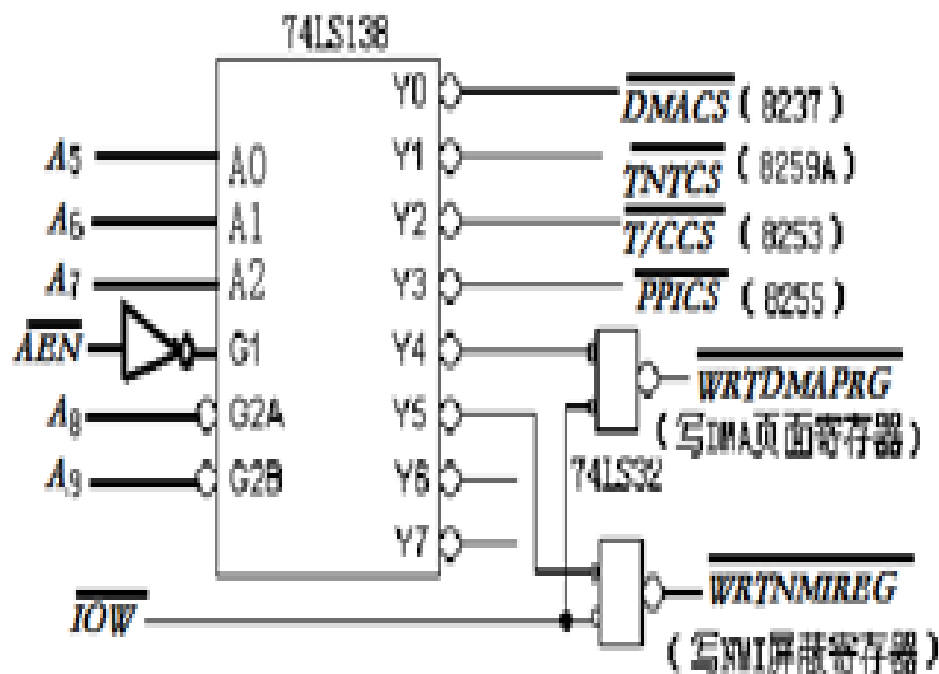
I/O接口



为区分不同的外设I/O口，需要地址译码电路产生低有效的信号（多数IC片选为低有效）分别接到各接口芯片的片选或使能端（即Chip Select/Enable）。

比如，地址总线的高位地址与CPU的控制信号，经译码电路产生I/O接口的片选信号，地址总线的低位地址不参与片选译码，直接连到接口芯片的地址端，对片内端口进行寻址。

PC系列微机中I/O端口译码电路



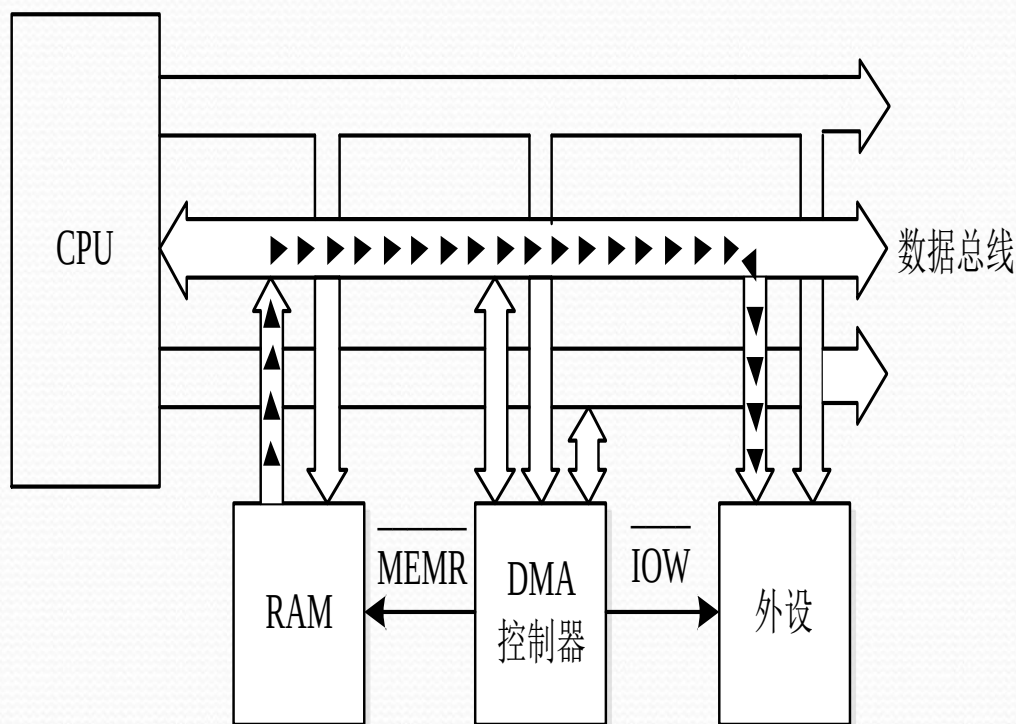
PC微机I/O地址线有16根，对应的I/O端口编址可达64K字节。

微机中有DMA控制器 (8237)、中断控制器 (8259A)、计数/定时器 (8253)、并行可编程接口芯片 (8255A) 等。

IBM—PC XT/AT系统主板上的I/O接口片选译码电路

【知识拓展】

DMA控制器

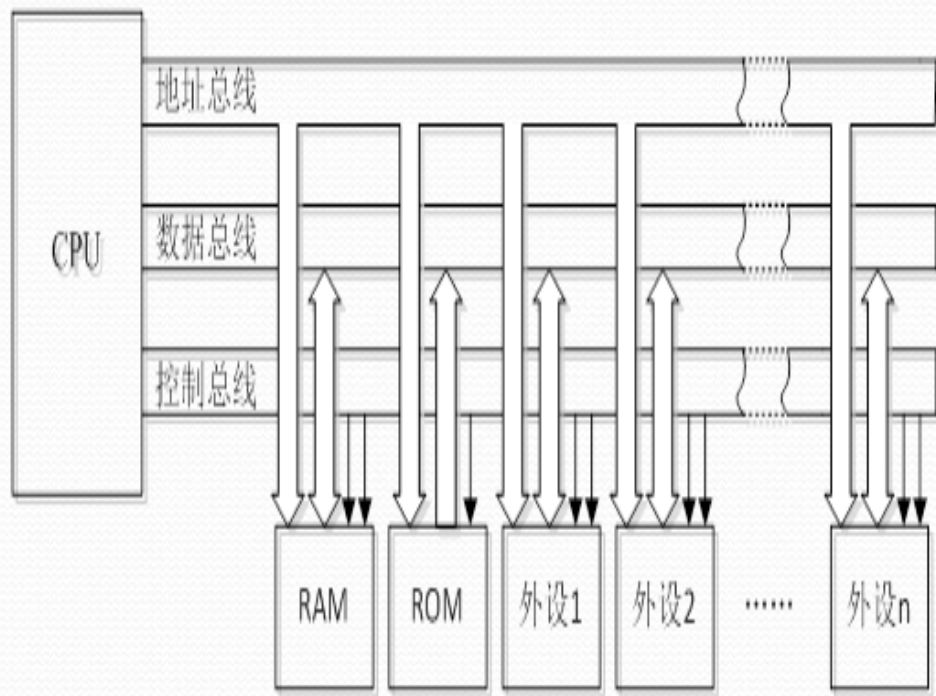


直接存储器存取是一种快速传送数据的机制。其特点是不经过CPU，不破坏CPU内各寄存器的内容，直接在存储器 and 外围设备端口间开辟高速的数据传送通路，进行数据交换，

因此，利用DMA进行数据存取时不需要CPU进行干预，可减轻CPU的负担，提高系统执行应用程序的效率。

查询和中断

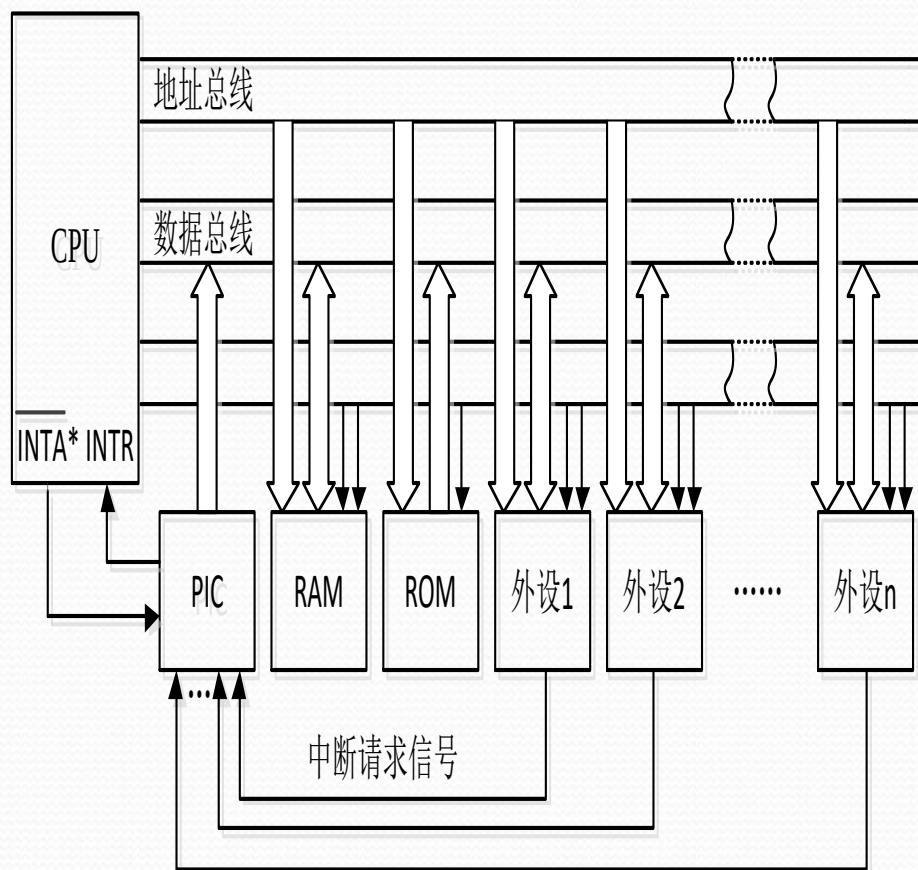
计算机中CPU需要经常为外设提供服务。例如，向外设发送数据或接受外设送来的数据。CPU可用2种方式获知外设是否有服务的需求：查询和中断。



查询： CPU需要主动地不断去循环检测每一个外设看其是否提出了服务的请求。通常，一个外设接口，如串行口内部寄存器中会有一个标志位用于表示外设是否有服务的请求提出，CPU可通过数据总线读取该位的信息以查询外设的需求。

缺点： 占用CPU资源，只能按照先查询先服务的方式。

查询和中断



中断： CPU只有在接收到一个中断请求信号后，才停止当前的工作，转去为外设提供服务。

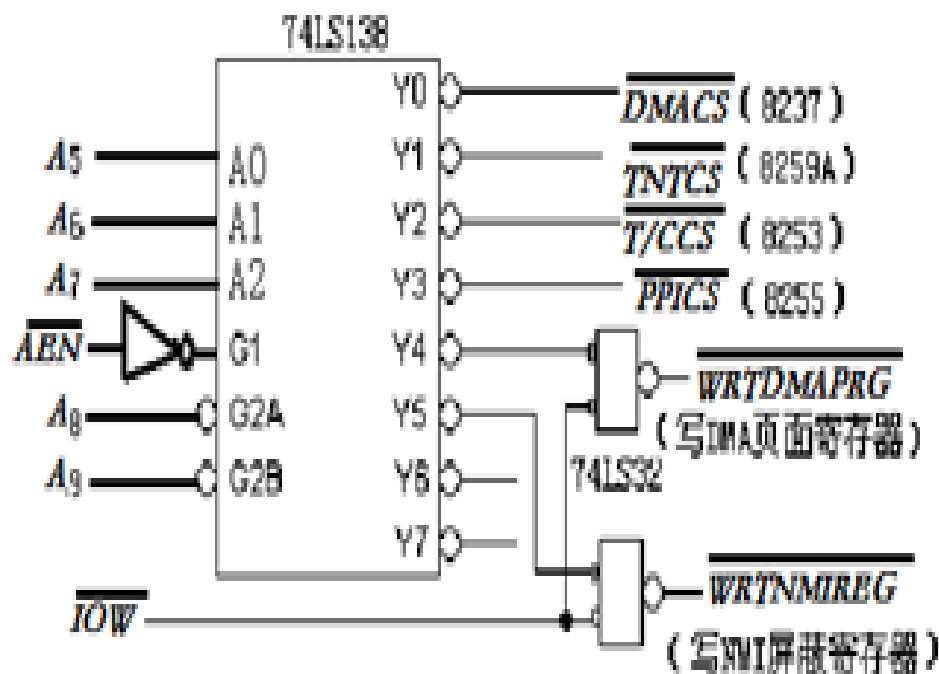
中断的处理由**可编程中断控制器**(Programmable Interrupt Controller, PIC)来管理。由PIC接收所有外设的中断请求，并判别中断的优先级。

PC系列微机中I/O端口译码电路

PC微机I/O地址线有16根，对应的I/O端口编址可达64K字节。

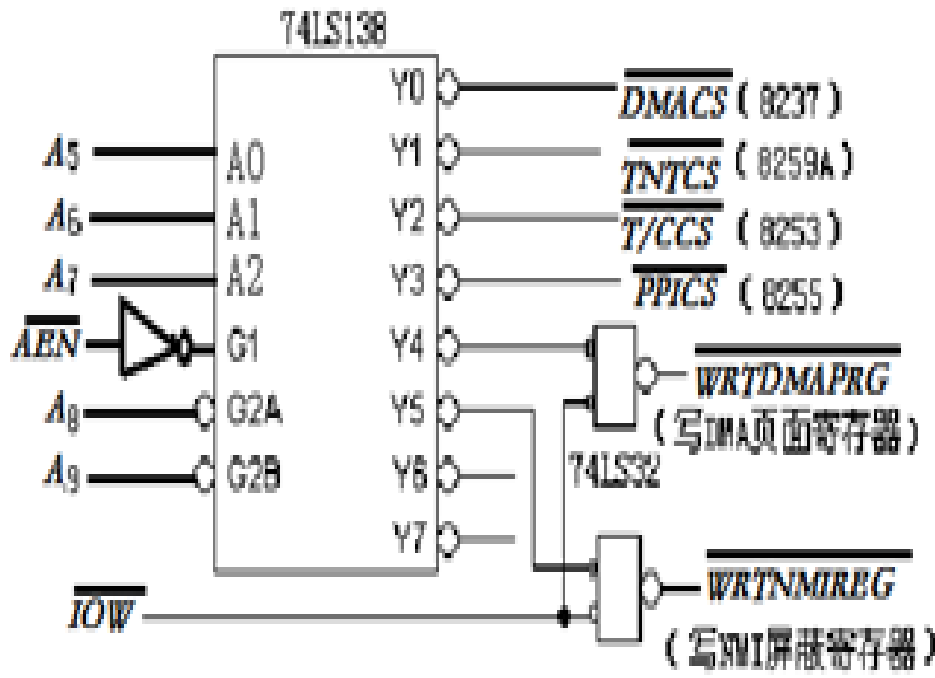
微机中有DMA控制器（8237）、中断控制器（8259A）、计数/定时器（8253）、并行可编程接口芯片（8255A）等。

以上某个器件工作时，*AEN*信号会处于低电平。



IBM—PC XT/AT系统主板上的I/O接口片选译码电路

PC系列微机中I/O端口译码电路



IBM—PC XT/AT系统主板上的I/O接口片选译码电路

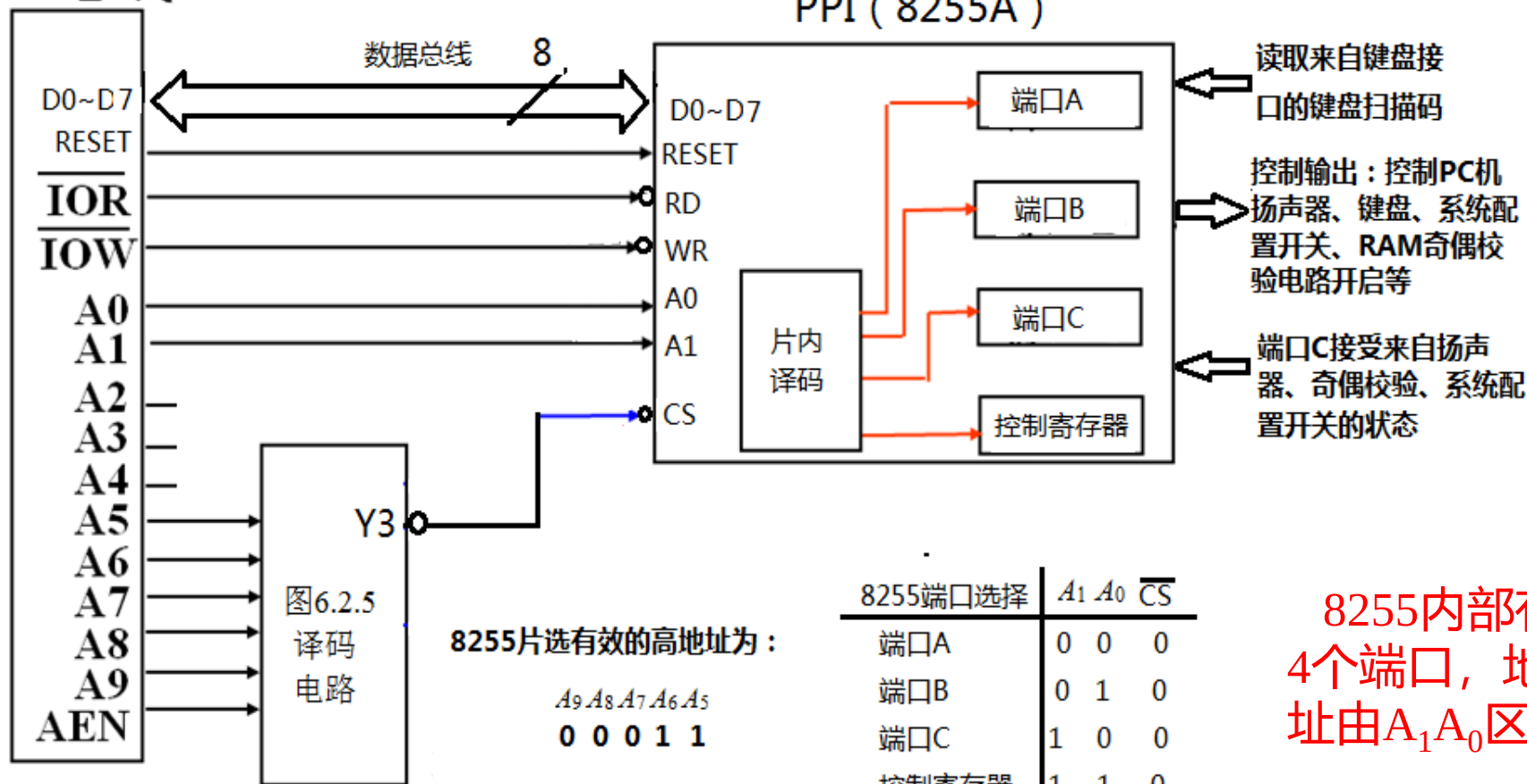
问题: 请确定8237、8259、8253、8255的端口地址?

地址线	对应地址范围	接口芯片
A9 A8 A7 A6 A5 A4 A3 A2 A1 A0		
0 0 0 0 0 × × × × ×	000H~01FH	DMA8237
0 0 0 0 1 × × × × ×	020H~03FH	中断控制器 8259A
0 0 0 1 0 × × × × ×	040H~05FH	定时计数器 8253
0 0 0 1 1 × × × × ×	060H~07FH	并行接口 8255
0 0 1 0 0 × × × × ×	080H~09FH	写 DMA 页面寄存器
0 0 1 0 1 × × × × ×	0A0H~0BFH	写 NMI 页面寄存器
0 0 1 1 0 × × × × ×	0C0H~0DFH	
0 0 1 1 1 × × × × ×	0E0H~0FFH	

NMI: Non Maskable Interrupt (不可屏蔽中断)

PC系列微机中I/O端口译码电路

PC总线



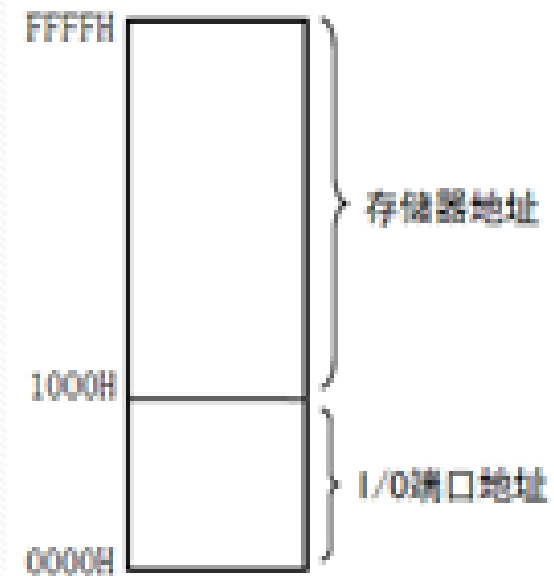
8255内部有4个端口，地址由 A_1A_0 区分。

接口芯片片内端口寻址

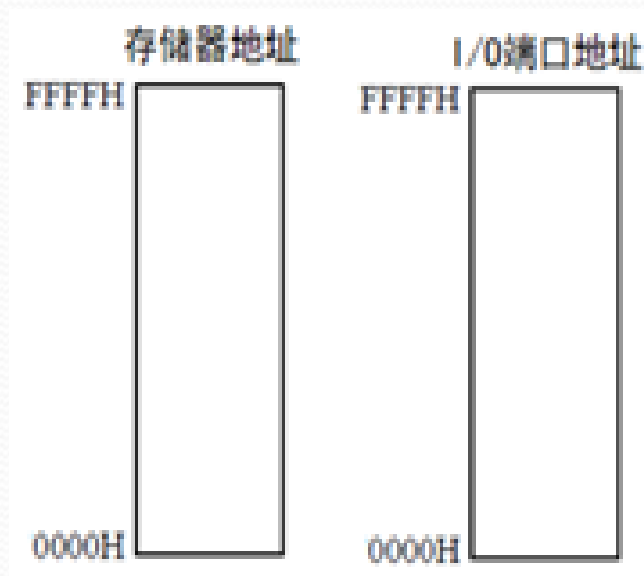
I/O端口的编址（或寻址）方式

微处理器除了访问I/O设备，还需要访问存储器。不同处理器对存储器和I/O接口的编址方法不同，主要有两种。

端口和存储器统一编址，即存储器映射方式：把每个存储单元视为一个外设，对应一个地址。



I/O和存储器分开独立编址，即I/O映射方式：地址空间重叠，靠指令区分。优点是I/O端口地址不占用存储器空间。

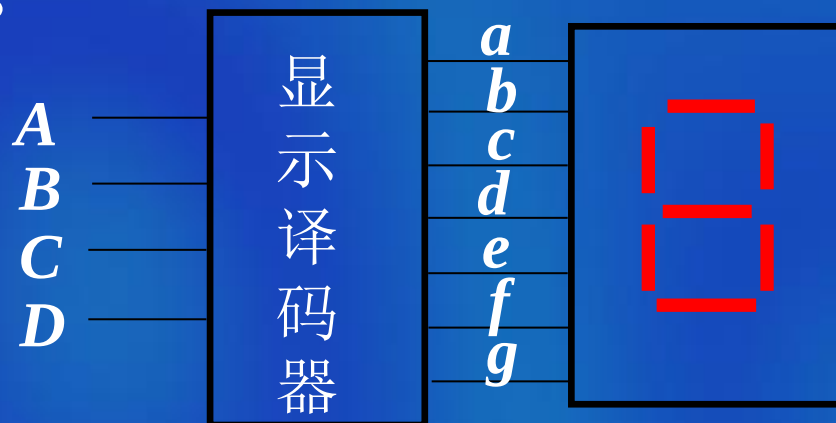


数码管和BCD - 七段显示译码器



数字系统中运行的是二进制数，但在数字测量仪表和各种显示系统中，常需将数字量用人们习惯的十进制字符直观地显示出来，这就要靠专门的译码电路把二进制数译成十进制字符，通过驱动电路由数码显示器显示出来。

在中规模集成电路中，常把译码和驱动电路集于一体，用来驱动数码显示管。



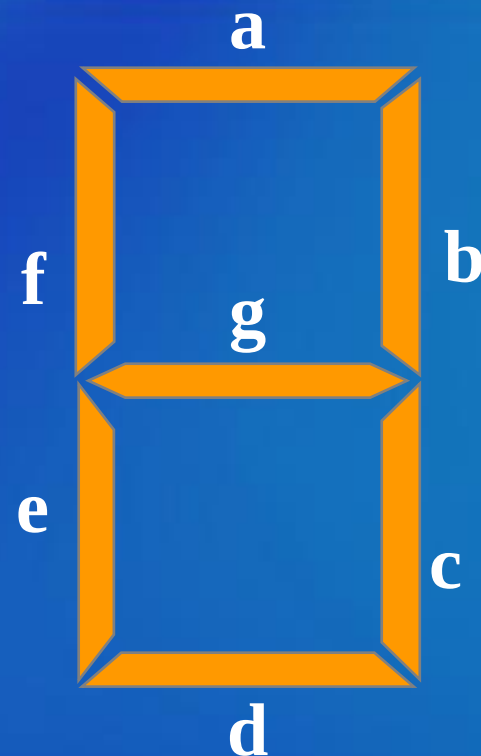
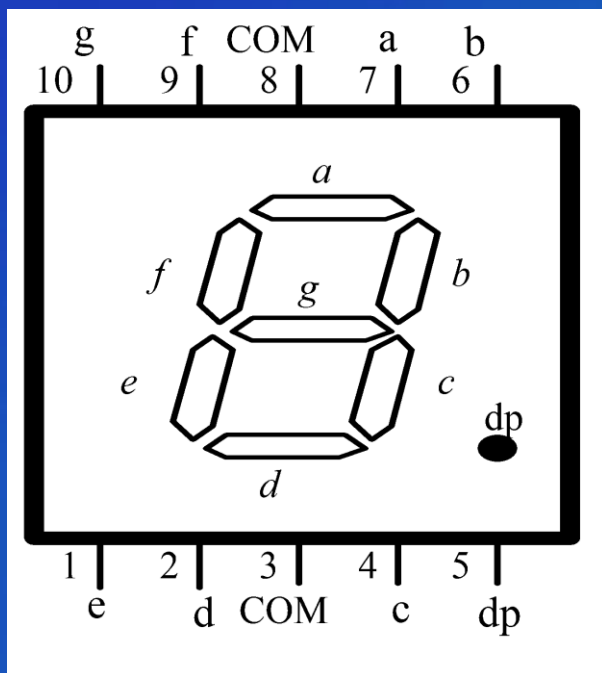
1. 七段数码管的结构及工作原理

数码管根据发光段数分为七段数码管和八段数码管。

发光段可以用： 荧光材料(称为荧光数码管);
发光二极管(称为LED数码管);

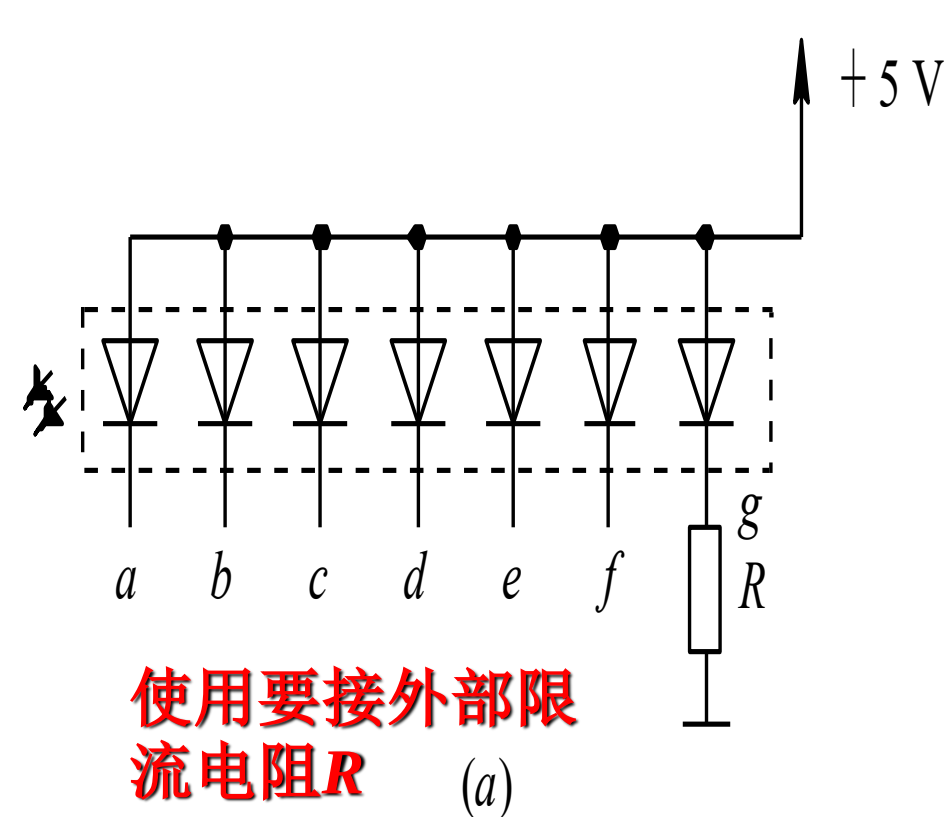


有七个发光段(a、b、c、d、e、f、g);



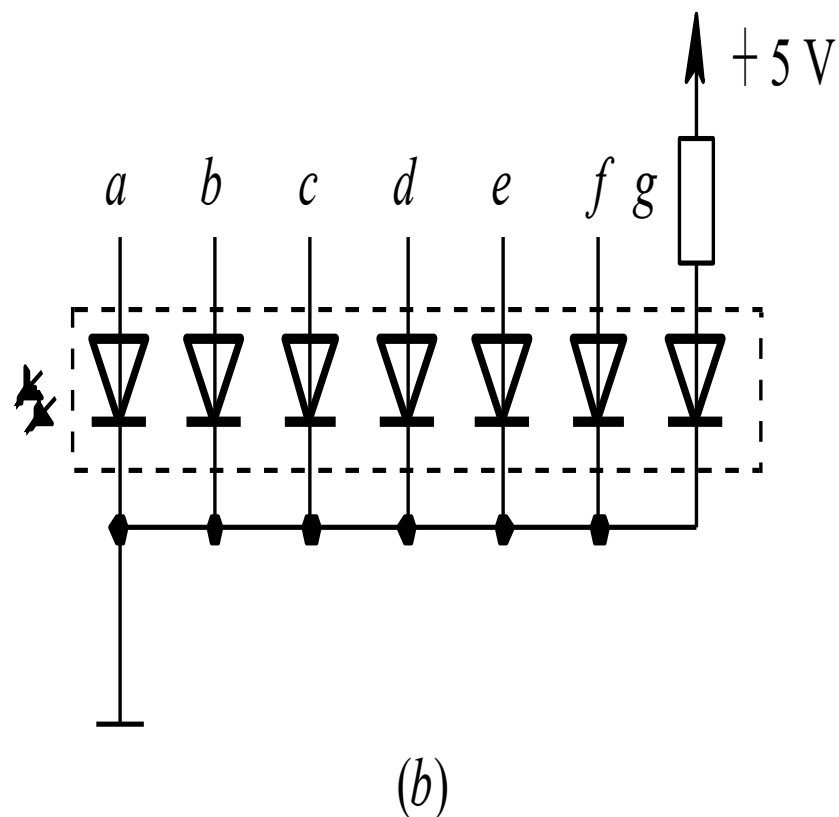
LED数码管的每个段为一个或数个发光二极管，加上适当的电压时，对应段就发光。

LED数码管的内部两种接法：



(a)

(a) 共阳极



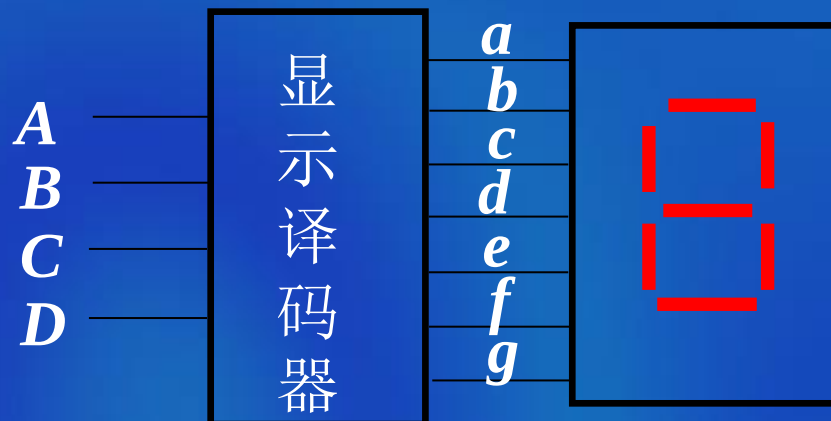
(b)

(b) 共阴极

显示译码器

BCD - 七段译码器

8421BCD码



(1) BCD七段译码器74LS47的符号图

输入 $A_3A_2A_1A_0$ 是四位BCD码

输出是驱动数码管工作的七段反码
码 $\sim$, 即低电平有效。

辅助控制信号输入端:

$\overline{LT}$ —— 试灯输入端, 用来测试七段数码管的好坏。

$\overline{BI}$ —— 熄灭信号输入端, 可控制数码管是否显示。

$\overline{RBO}$ —— 灭零信号输出端。

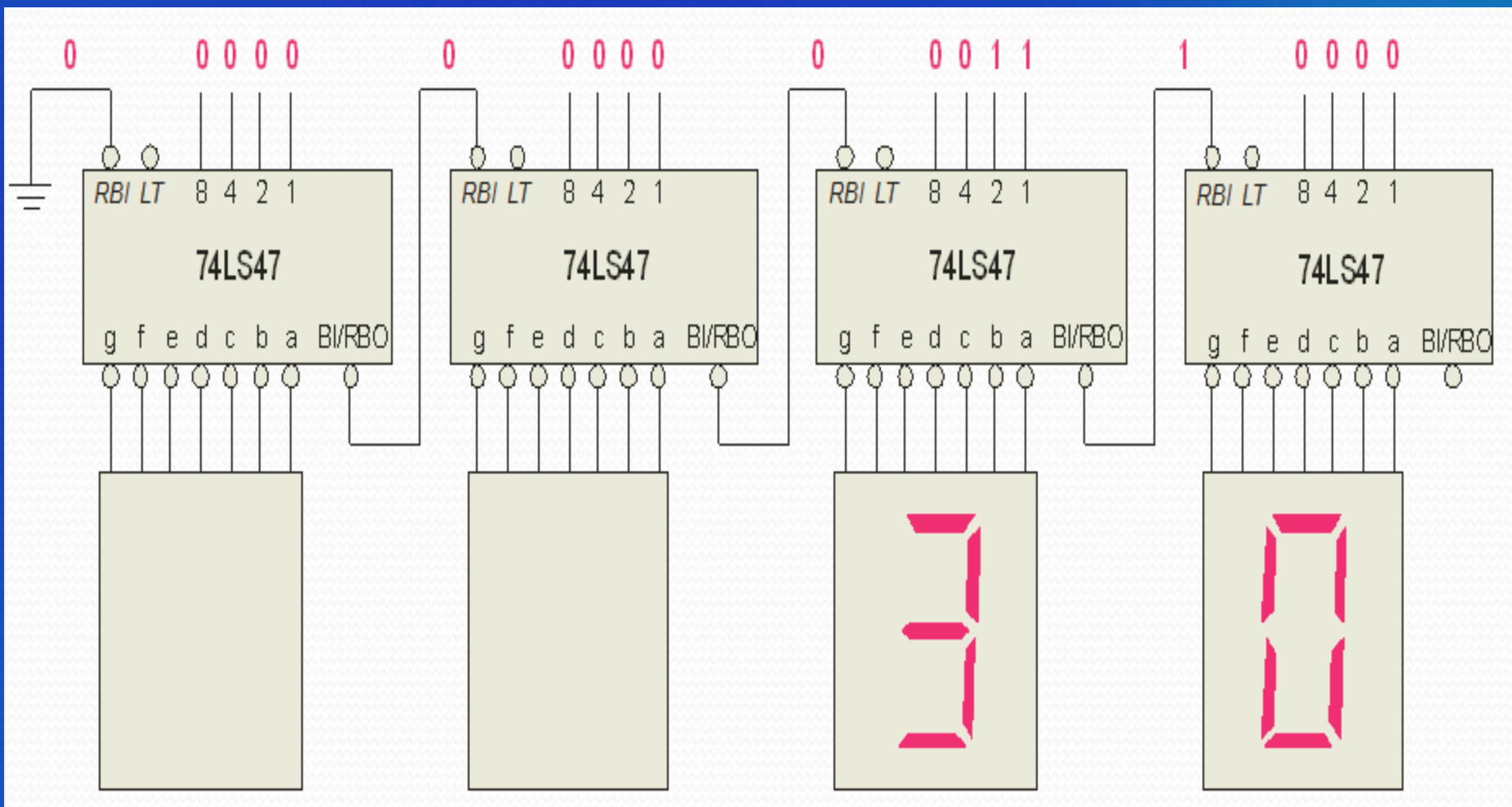
$\overline{RBI}$ —— 灭零输入端, 用来熄灭不需要显示的0。



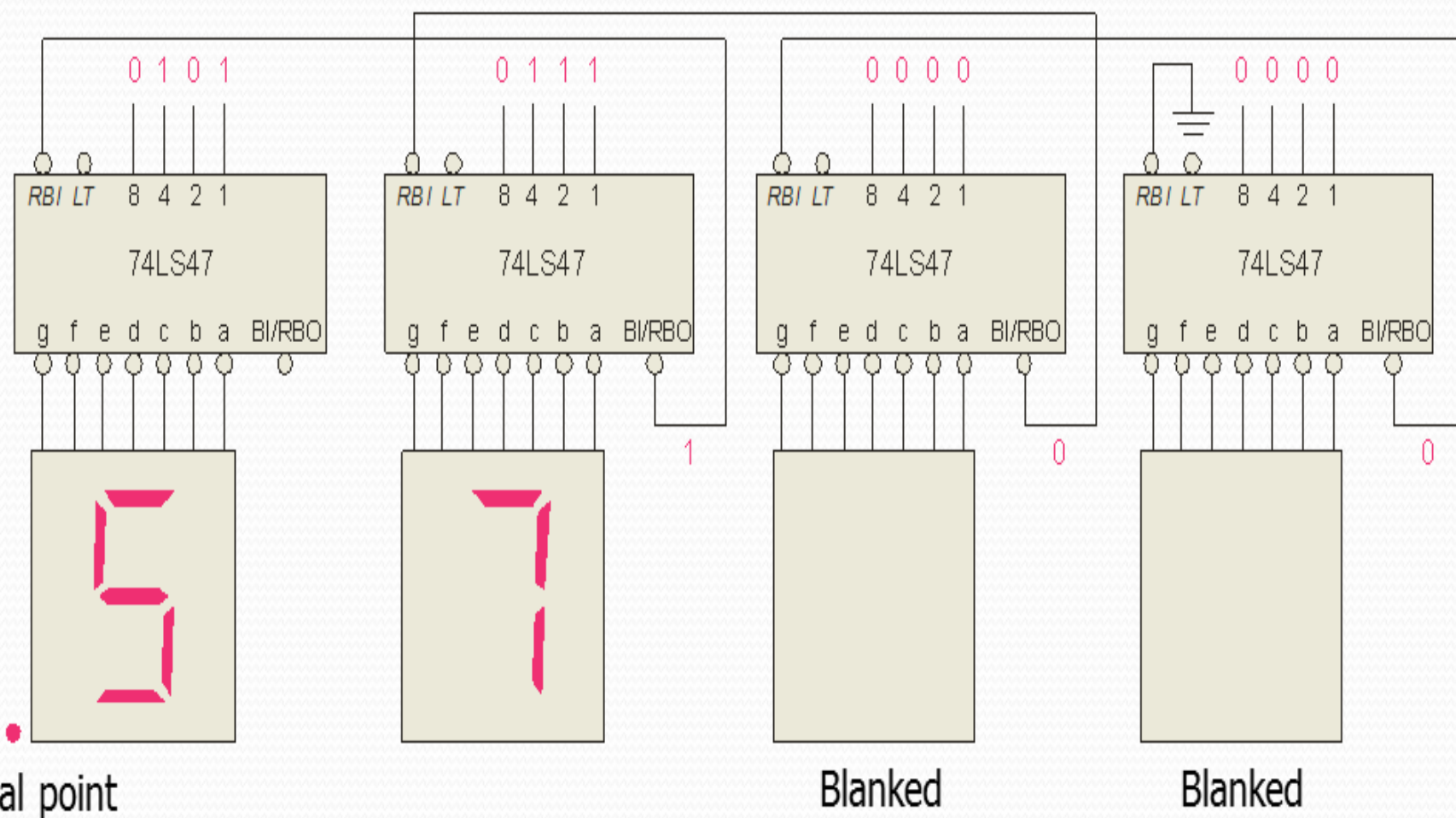
BCD七段译码器74LS47功能表

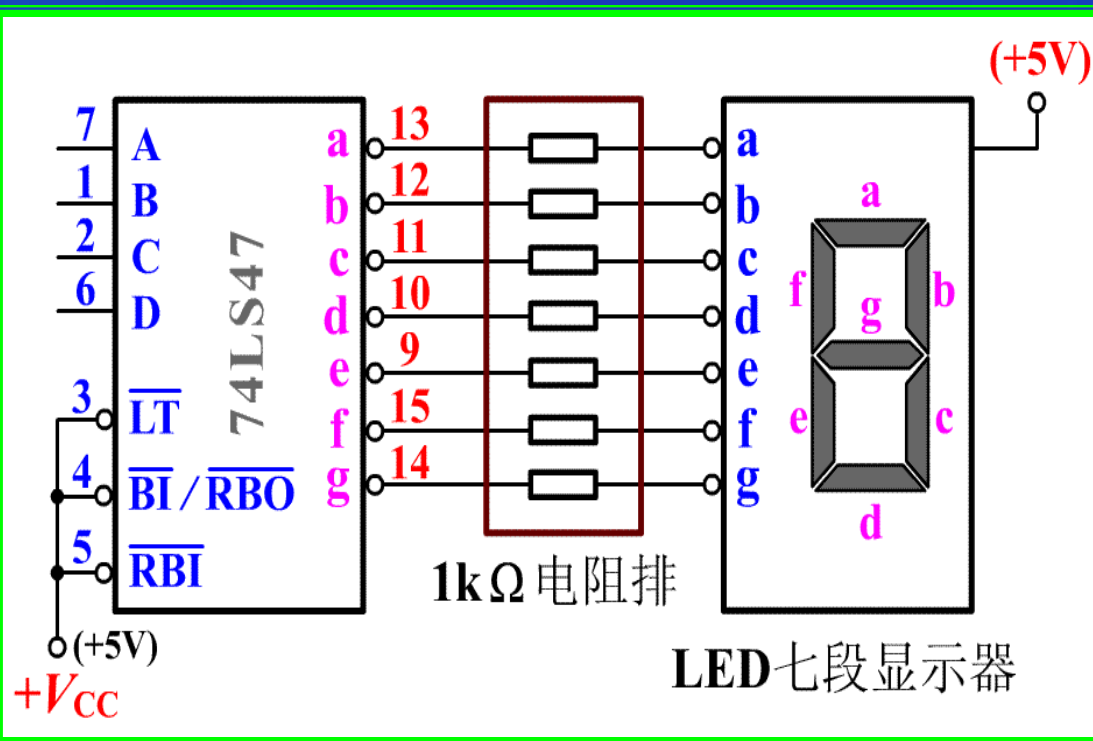
十进制数字或功能	输 入						输 出								显示字型
	$\overline{LT}$	$\overline{RBI}$	A_3	A_2	A_1	A_0	$\overline{BI} / \overline{RBO}$	$\overline{a}$	$\overline{b}$	$\overline{c}$	$\overline{d}$	$\overline{e}$	$\overline{f}$	$\overline{g}$	
0	1	1	0	0	0	0	1	0	0	0	0	0	0	1	0123456789
1	1	×	0	0	0	1	1	1	0	0	1	1	1	1	
2	1	×	0	0	1	0	1	0	0	1	0	0	1	0	
3	1	×	0	0	1	1	1	0	0	0	0	1	1	0	
4	1	×	0	1	0	0	1	1	0	0	1	1	0	0	
5	1	×	0	1	0	1	1	0	1	0	0	1	0	0	
6	1	×	0	1	1	0	1	1	1	0	0	0	0	0	
7	1	×	0	1	1	1	1	0	0	0	1	1	1	1	
8	1	×	1	0	0	0	1	0	0	0	0	0	0	0	
9	1	×	1	0	0	1	1	0	0	0	1	1	0	0	
10	1	×	1	0	1	0	1	1	1	1	0	0	1	0	0123456789
11	1	×	1	0	1	1	1	1	1	0	0	1	1	0	
12	1	×	1	1	0	0	1	1	0	1	1	1	0	0	
13	1	×	1	1	0	1	1	0	1	1	0	1	0	0	
14	1	×	1	1	1	0	1	1	1	1	0	0	0	0	
15	1	×	1	1	1	1	1	1	1	1	1	1	1	1	
$\overline{BI}$	×	×	×	×	×	×	0	1	1	1	1	1	1	1	熄灭
$\overline{RBI}$	1	0	0	0	0	0	0	1	1	1	1	1	1	1	熄灭
$\overline{LT}$	0	×	×	×	×	×	1	0	0	0	0	0	0	0	试灯

➤ 熄灭前导零.



➤ 熄灭后导零.





LED七段显示器和译码驱动电路的连接实例。

LED七段显示器的驱动电路是由74LS47译码器、1kΩ的双列直插限流电阻排、七段共阳极LED显示器组成的。

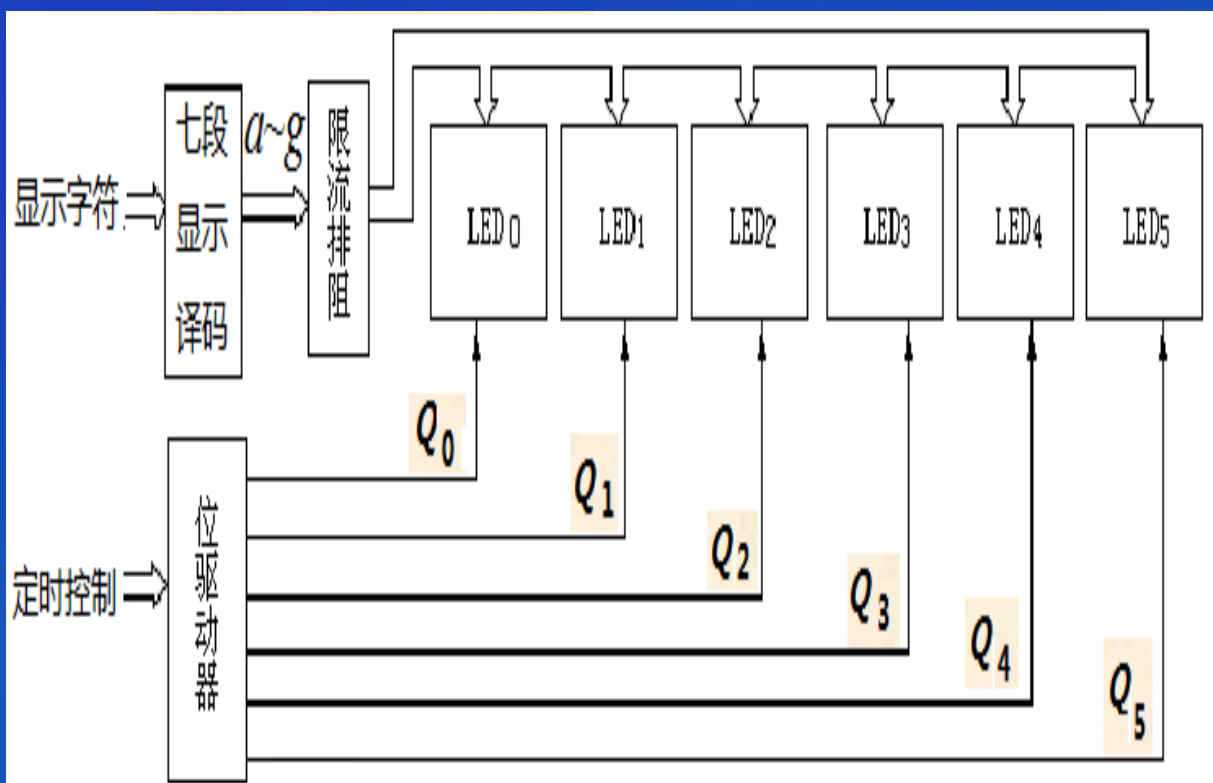
74LS47驱动电流达24mA，驱动数码管时需要外加限流电阻。

注意:

- a. 当选用共阳极LED数码管时, 应使用低电平有效的七段译码器驱动(如7446、7447); DM74LS47 $I_{OL}=24\text{mA}$
- b. 当选用共阴极LED数码管时, 应使用高电平有效的七段译码器驱动(如7448、7449)。
- c. 通常1英寸以上的显示器每个发光段由多个二极管组成, 需要较大的驱动电流。
- d. TTL集成电路的低电平驱动能力比高电平驱动能力大得多, 所以常用低电平有效的七段译码器。

LED数码管动态显示

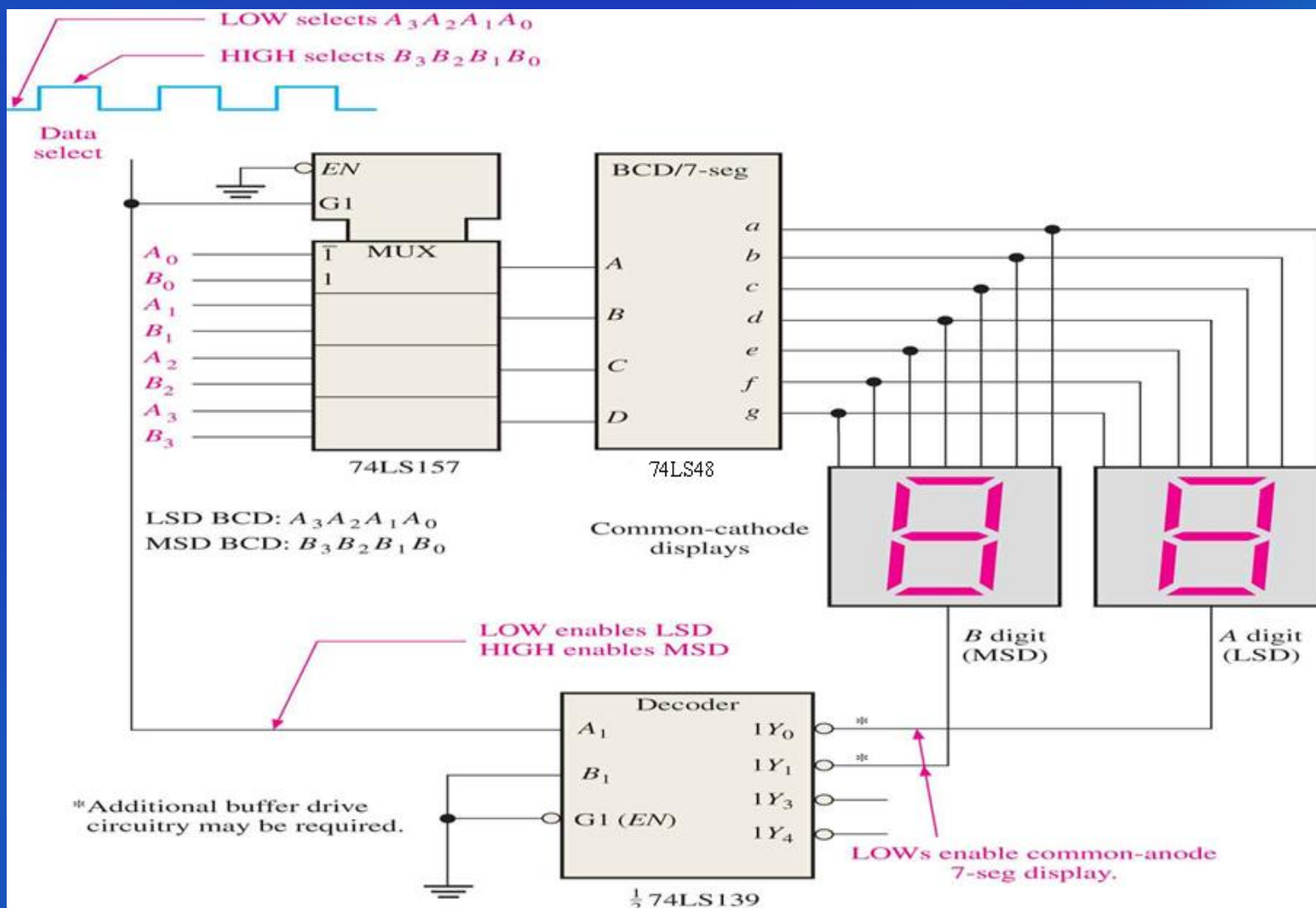
在微处理器系统中，多位LED显示一般采用动态扫描方式、分时循环显示，即多个发光管轮流交替点亮。



数码管段信号端a~g分别接在一起，控制它们的公共端**分时轮流有效**。

利用人眼的**视觉滞留现象**，只要在1秒内一个数码管亮24次以上，每次点亮时间维持2ms以上，则人眼感觉不到闪烁，宏观上仍可看到多位LED同时显示的效果。

LED数码管 (共阴极) 动态显示



显示时，先送段码，将同时加到2个LED上，之后，送位码，只有共阴极引脚得到低电平的数码管上才有显示。可见，通过段码和位码的控制，可将不同的字符依次显示在不同的数码管上。同一时刻实际只有1个数码管显示，但由于“视觉暂留”，只要刷新频率足够快（>24Hz），即可得到稳定显示。

编码器

(1) 二进制编码

将二进制数码(0或1)按一定规则组成代码表示一个特定对象，称为二进制编码。

(2) 编码器(Encoder)

具有编码功能的电路称为编码电路，而相应的MSI芯片称为编码器(Encoder)。

(3) 编码器的分类

按照被编对象的不同特点和编码要求，有各种不同的编码器，如二进制编码器、优先编码器和8421BCD编码器等。

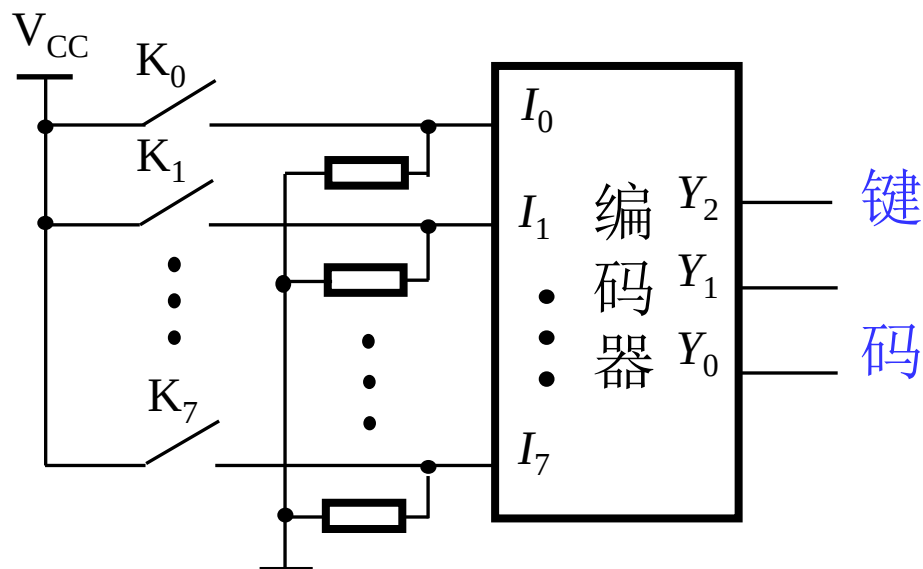
1. 二进制编码器

用 n 位二进制代码对 $N=2^n$ 个信号进行编码的电路，叫做二进制编码器。

二进制编码器也称之为 $2^n - n$ 线二进制编码器。

二进制编码器

有一键盘输入电路，一共有8个按键，键按下时，对应的输入信号为高电平。



编码器的作用就是把每一个按键发出的这个高电平信号转化成相应的编码（键码/键值）。

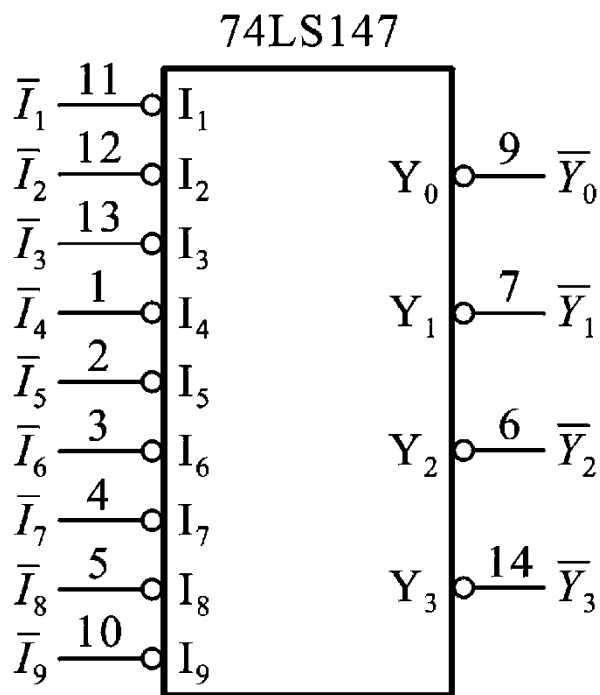
2. 优先编码器

与普通编码器不同，优先编码器允许多个输入信号同时有效，但它只按其中优先级别最高的有效输入信号编码，对级别较低的输入信号不予理睬。

常用的MSI优先编码器有10线—4线(如74LS147)、8线—3线(如74LS148)。

(1) 优先编码器74LS147

a. 74LS147符号图



输入是10路 ($\bar{I}_0 \sim \bar{I}_9$, $\bar{I}_0$ 隐含其中)
被编对象。

a. 框外输入端的小圆圈表示输入信号
低电平(逻辑0)有效。

b. 输出端的小圆圈表示反码输出。

c. 在框外小圆圈对应的输入和输出
变量上冠以 “ $\bar{}$ ”号与之对应。这样
输入输出端的小圆圈可以理解为逻辑
非运算。

b. 编码器74LS147功能表

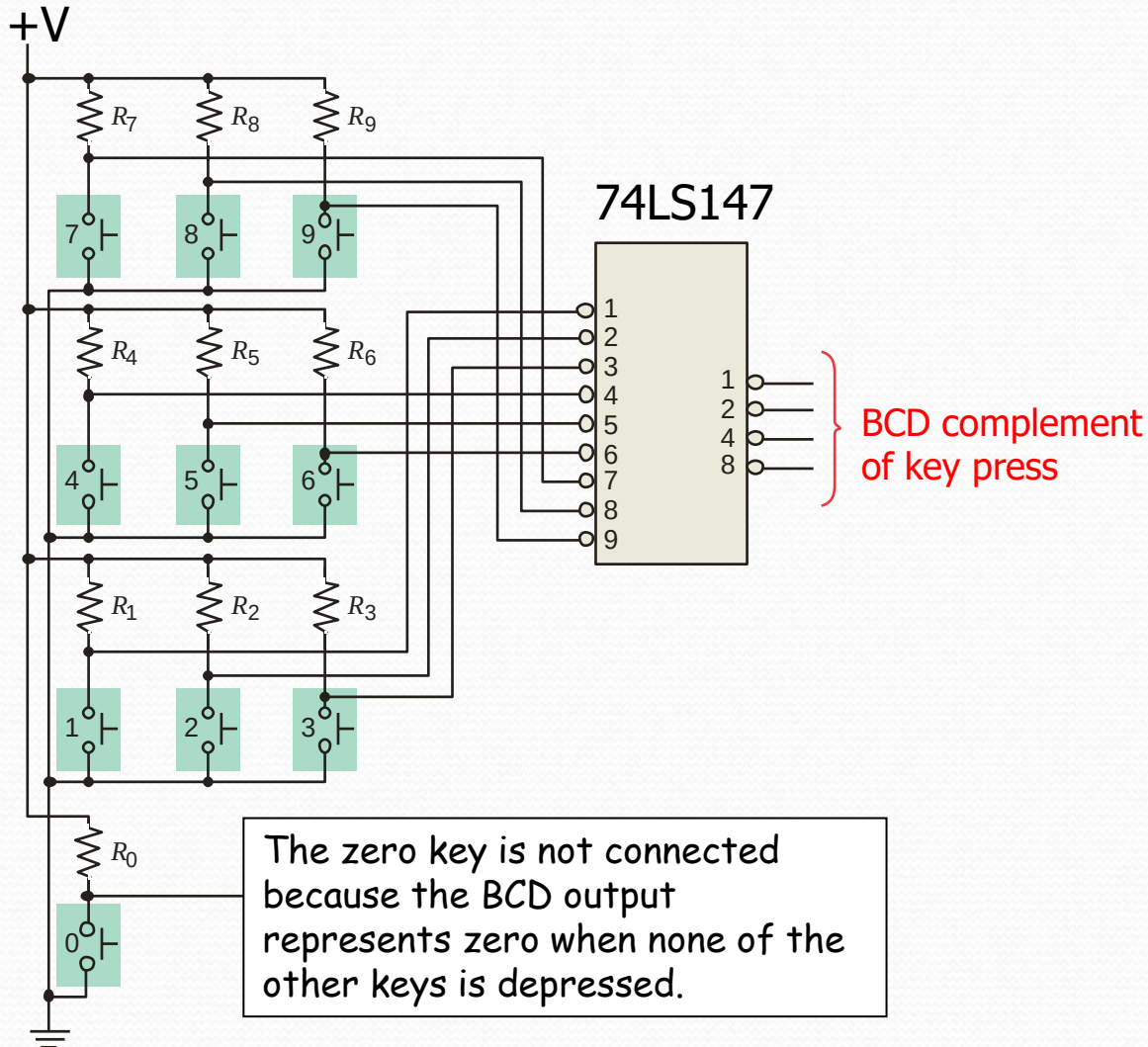
输 入										输 出			
$\bar{I}_1$	$\bar{I}_2$	$\bar{I}_3$	$\bar{I}_4$	$\bar{I}_5$	$\bar{I}_6$	$\bar{I}_7$	$\bar{I}_8$	$\bar{I}_9$		$\bar{Y}_3$	$\bar{Y}_2$	$\bar{Y}_1$	$\bar{Y}_0$
×	×	×	×	×	×	×	×	0		0	1	1	0
×	×	×	×	×	×	×	0	1		0	1	1	1
×	×	×	×	×	×	0	1	1		1	0	0	0
×	×	×	×	×	0	1	1	1		1	0	0	1
×	×	×	×	0	1	1	1	1		1	0	1	0
×	×	×	0	1	1	1	1	1		1	0	1	1
×	×	0	1	1	1	1	1	1		1	1	0	0
×	0	1	1	1	1	1	1	1		1	1	0	1
0	1	1	1	1	1	1	1	1		1	1	1	0

输入是10路被编对象，允许同时有几个输入端送入编码信号。

$\bar{I}_9$ 优先权最高，依次降低， $\bar{I}_0$ 优先权最低。当 $\bar{I}_9=0$ 时，无论其他输入端有无信号(以×表示)，输出端只给出 $\bar{I}_9$ 反码形式的编码，即0110。当 $\bar{I}_9=1$ ， $\bar{I}_8=0$ 时，无论其他输入端有无信号，只对 $\bar{I}_8$ 编码，输出其反码形式 0111，……。

当所有输入端都为1时，对 $\bar{I}_0$ 进行编码，输出其反码形式 1111。

An application of 74LS147

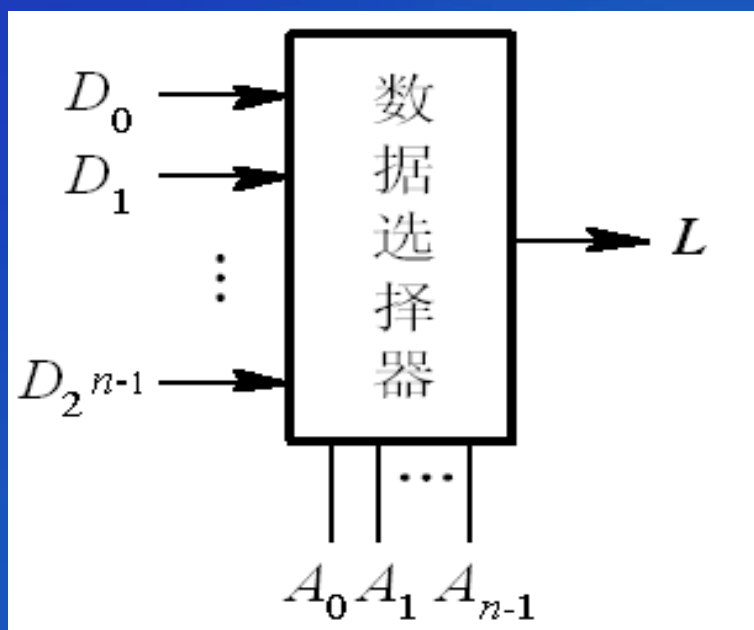


DECIMAL DIGIT (inputs)	BCD complement $\overline{A_3} \overline{A_2} \overline{A_1} \overline{A_0}$
1	1110
2	1101
3	1100
4	1011
5	1010
6	1001
7	1000
8	0111
9	0110

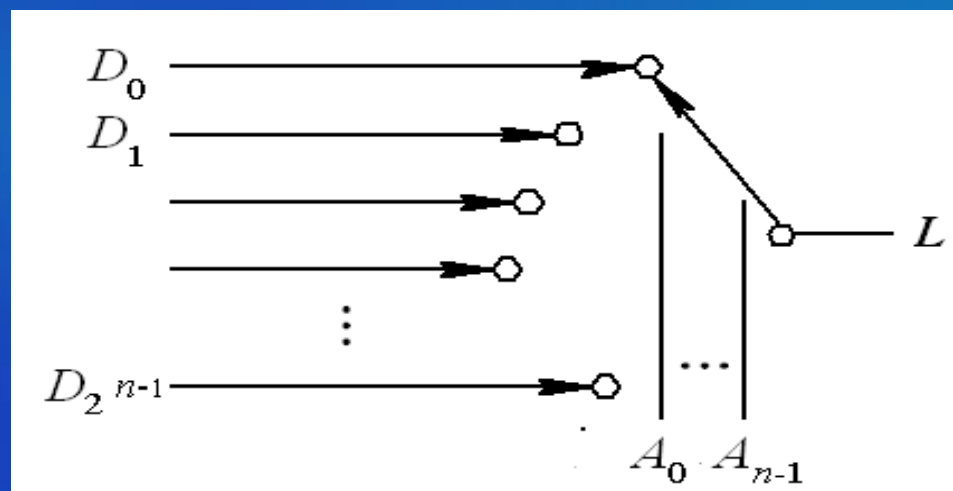
7.3 多路选择器

多路选择器(MUX)

在数字系统中，有时需要将多路数字信息分时地从一条通道传送，完成这一功能的电路称为多路数据选择器(Multiplexer，简称MUX)。



n 位地址输入、 2^n 位数据输入、1位输出。



在地址输入的控制下，从多路输入数据中选择一路输出。功能类似于一个单刀多掷开关。常有2选1、4选1、8选1、16选1等。

中规模双4选1数据选择器74LS253

两个完全相同的4选1数据选择器构成。

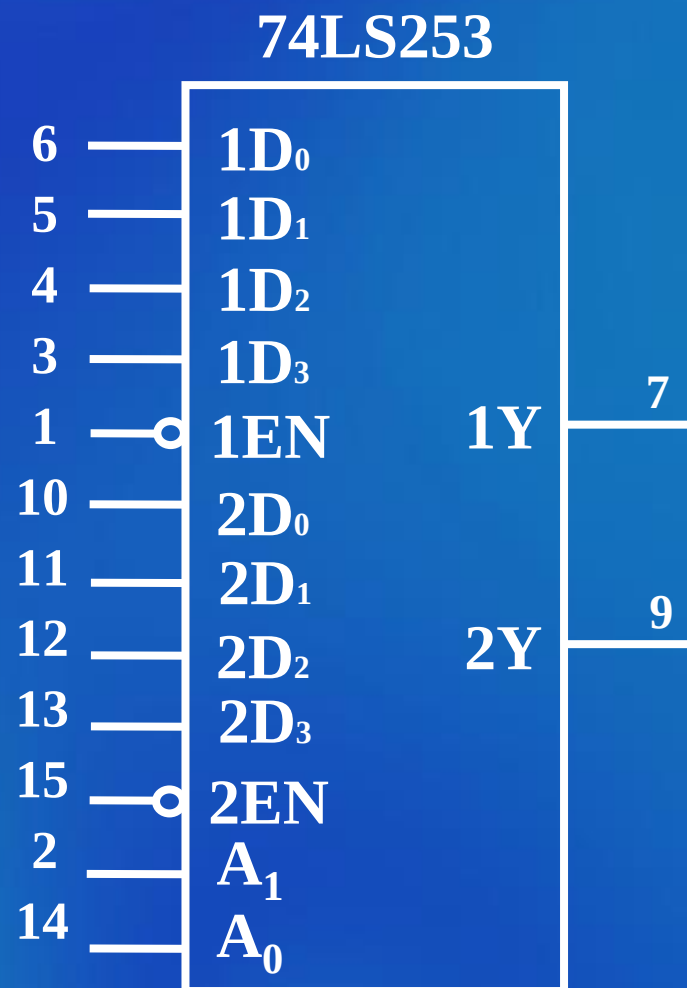
$1D_0 \sim 1D_3$ 、 $2D_0 \sim 2D_3$ 是两组独立的数据输入端；

$1Y$ 、 $2Y$ 分别为两组独立的输出端； A_1 和 A_0 是共用地址输入端。

$1\overline{EN}$ 和 $2\overline{EN}$ 分别是两路选通输入端。

选通信号等于 '1' 时，被禁止。

选通信号为 '0' 时，把与地址码相应的一路数据选送到输出端。



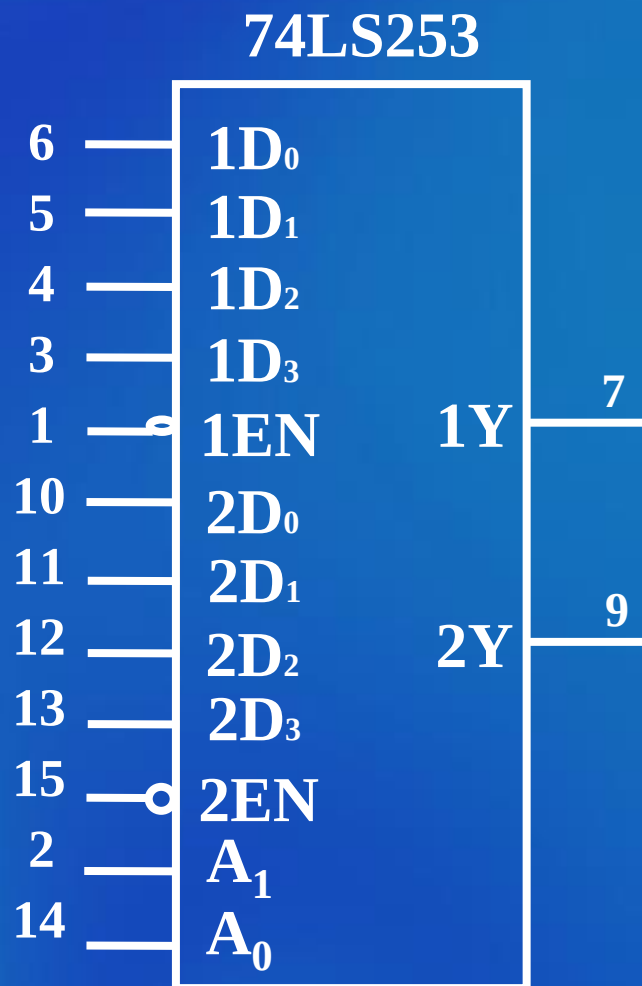
c. 功能表

地址		选通	数据	输出
A_1	A_0	$\overline{EN}$	D	Y
×	×	1	×	(Z)
0	0	0	$D_0 \sim D_3$	D_0
0	1	0	$D_0 \sim D_3$	D_1
1	0	0	$D_0 \sim D_3$	D_2
1	1	0	$D_0 \sim D_3$	D_3

当使能端有效时，输出表示为：

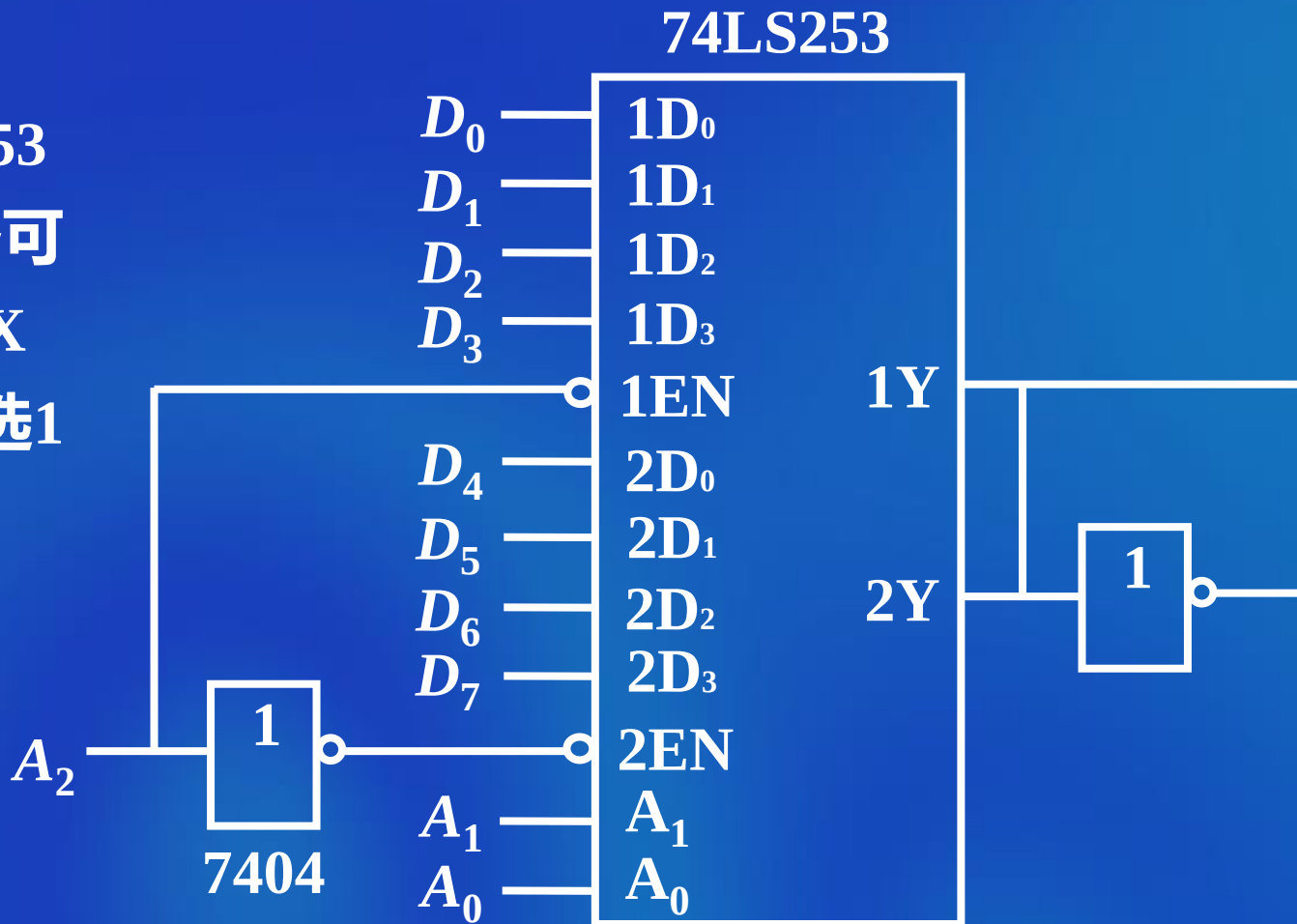
$$Y = D_0 \overline{A_1} \overline{A_0} + D_1 \overline{A_1} A_0 + D_2 A_1 \overline{A_0} + D_3 A_1 A_0$$

$$Y = \sum_{i=0}^{2^n-1} m_i D_i \quad \text{n为地址端个数； } m_i \text{ 是地址输入端的最小项； } D_i \text{ 表示对应的数据输入端。}$$

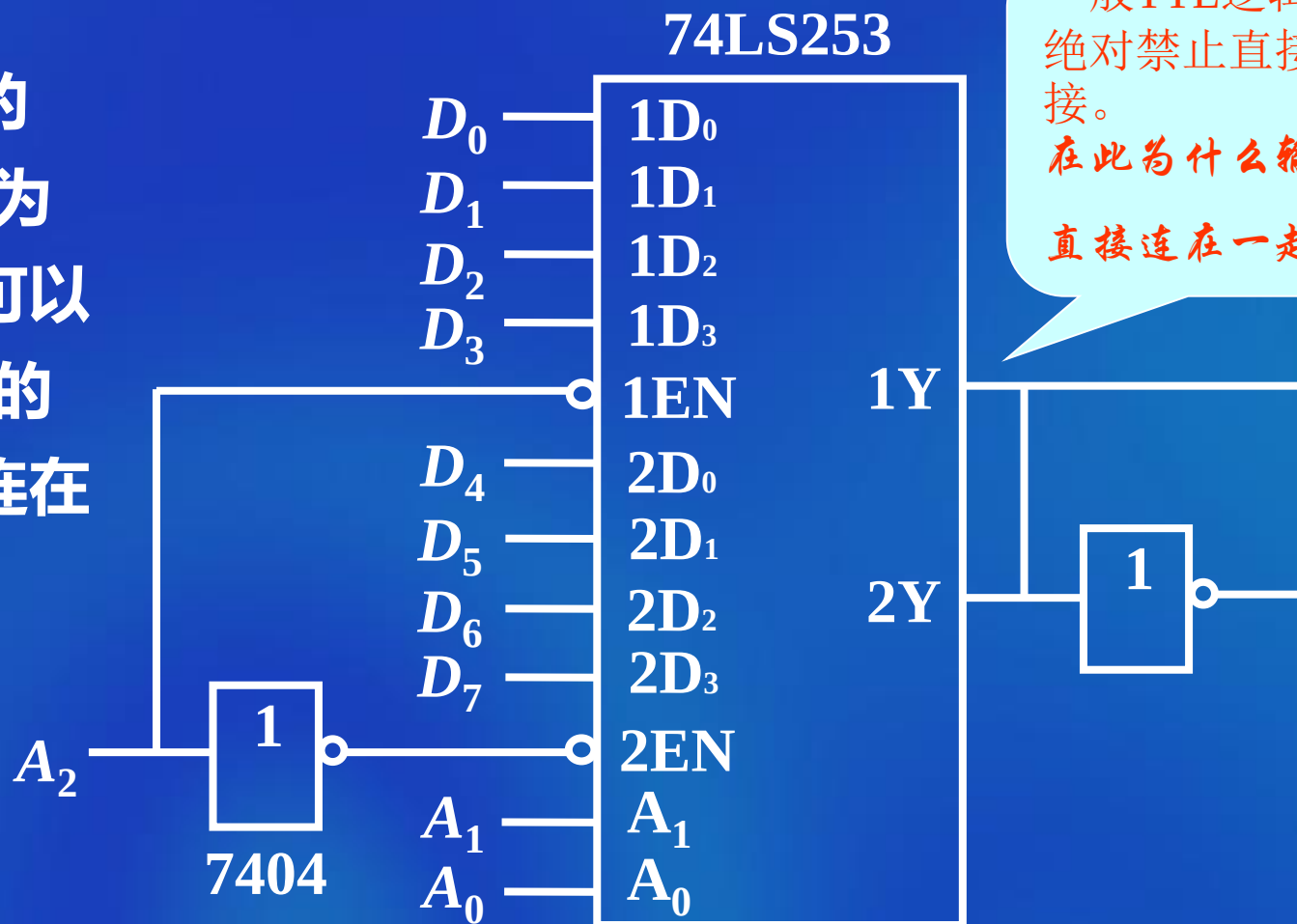


1. MUX的扩展

a. 一片74LS253
和若干门电路可
将双4选1MUX
扩展为一个8选1
的MUX。



由于未选通的
MUX输出端为
高阻，因此可以
将两个MUX的
输出端直接连在
一起。

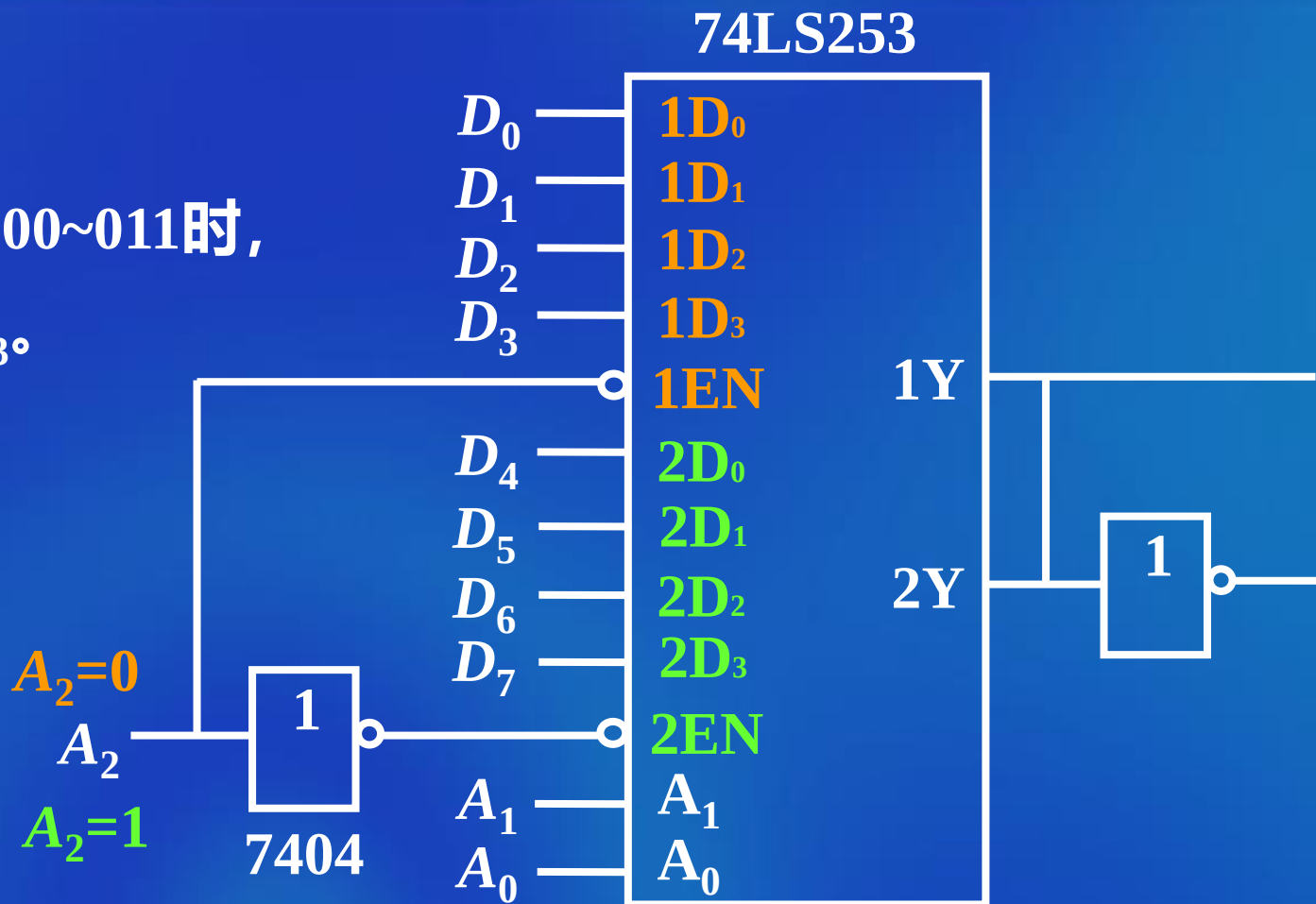


一般TTL逻辑输出
绝对禁止直接相
接。
在此为什么输出
直接连在一起？

74LS251、74LS253、74LS257是三态MUX。

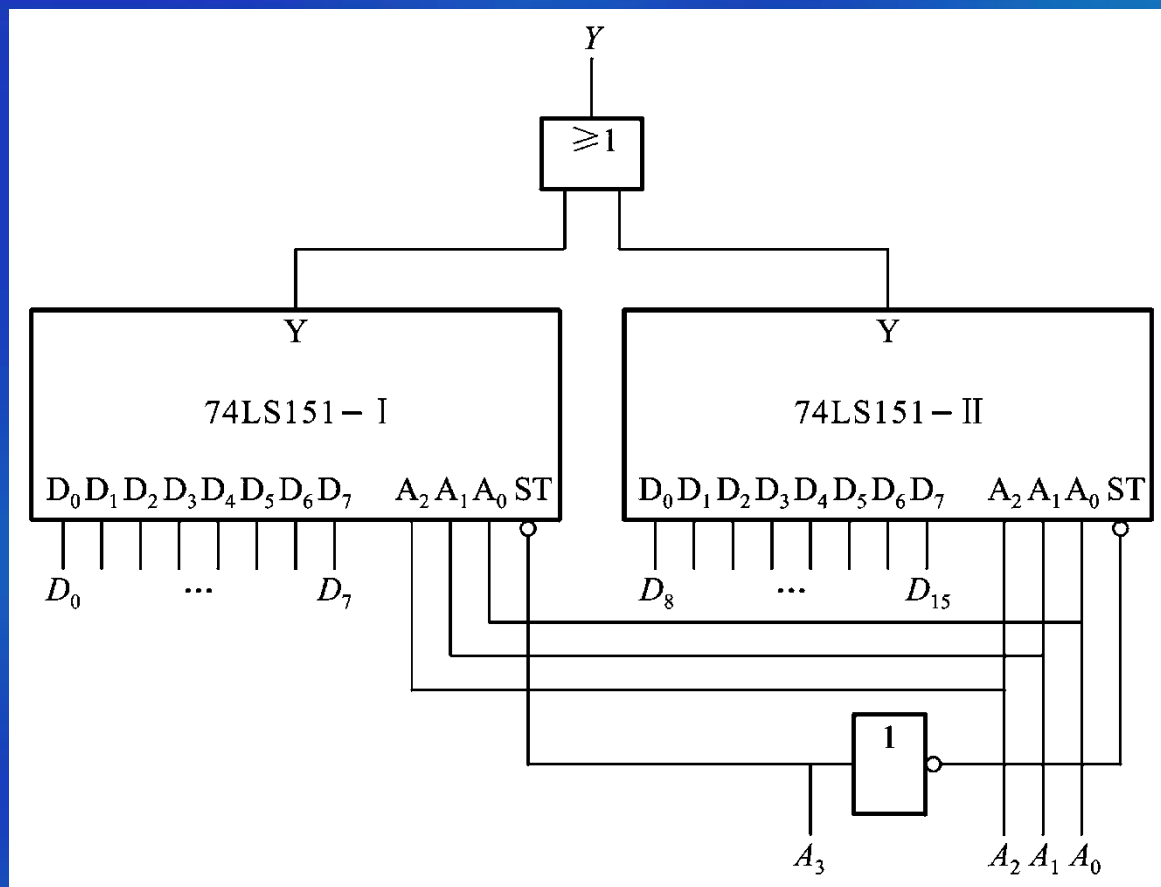
由图可知：

当 $A_2A_1A_0$ 为000~011时，
选通 $1D_0 \sim 1D_3$ 。



当 $A_2A_1A_0$ 为100~111时，
选通 $2D_0 \sim 2D_3$ 。

b .用两块8选1
数据选择器
74LS151构成16
选1数据选择器



选择器74LS151被禁止时，输出为零而非高阻状态。因此，输出不能直接相联。

2. 数据选择器的典型应用

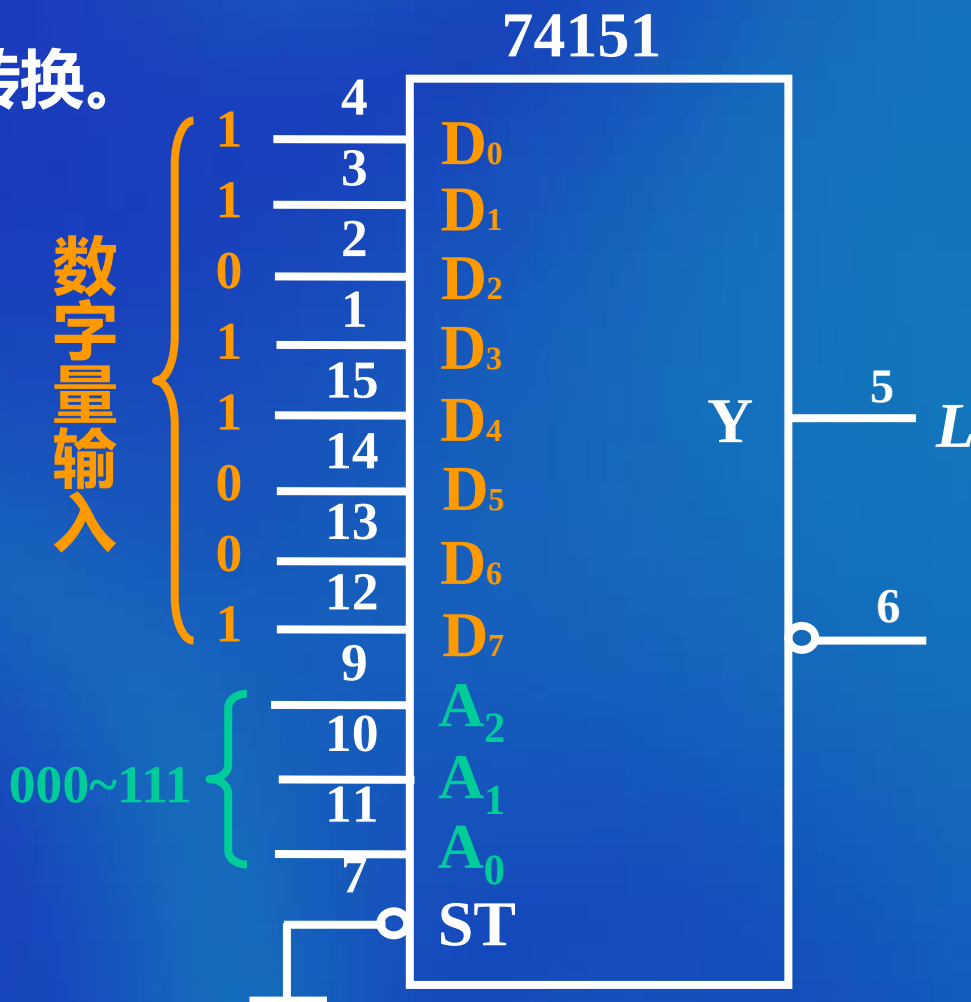
- (1) 作数据选择，以实现多路信号分时传送。
- (2) 实现组合逻辑函数。
- (3) 在数据传输时实现并—串转换。
- (4) 产生序列信号。

(1) 在数据传输时实现并—串转换。

当 $A_2A_1A_0$ 由000~111循环变化时，8位并行输入数据依次传送到输出端，被转换为串行数据输出。

输入数据 $D_0\sim D_7$ 为11011001

在输出端得到的串行数据波形



(2) 实现单输出函数

对于 n 个地址输入的MUX, 其表达式为

$$Y = \sum_{i=0}^{2^n-1} m_i D_i$$

其中 m_i 是由地址变量 A_{n-1} 、...、 A_1 、 A_0 组成的地址最小项。

而任何一个具有 l 个输入变量的逻辑函数都可以用最
小项之和来表示, 即

$$L = \sum_{i=0}^{2^l-1} m_i$$

这里的 m_i 是由函数的输入变量 A 、 B 、 C 、...组成的最小项。

只要将逻辑函数的输入变量 A 、 B 、 C 、...加至数据选择器地址输入端, 并适当选择 D_i 的值, 使 $L=Y$, 就可以用MUX实现函数 L 。

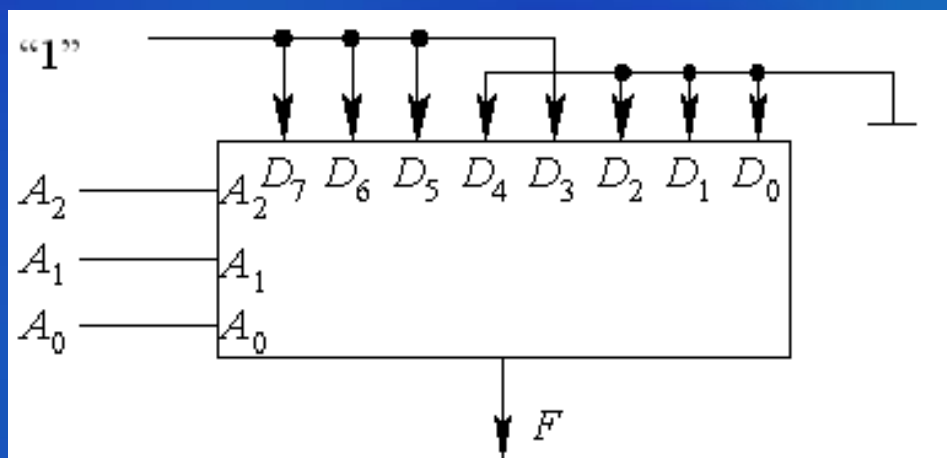
[例1] 用八选一数据选择器实现三变量多数表决器。

[解] 三变量多数表决器真值表及八选一数据选择器功能表。则

$$D_0 = D_1 = D_2 = D_4 = 0$$

$$D_3 = D_5 = D_6 = D_7 = 1$$

逻辑电路图

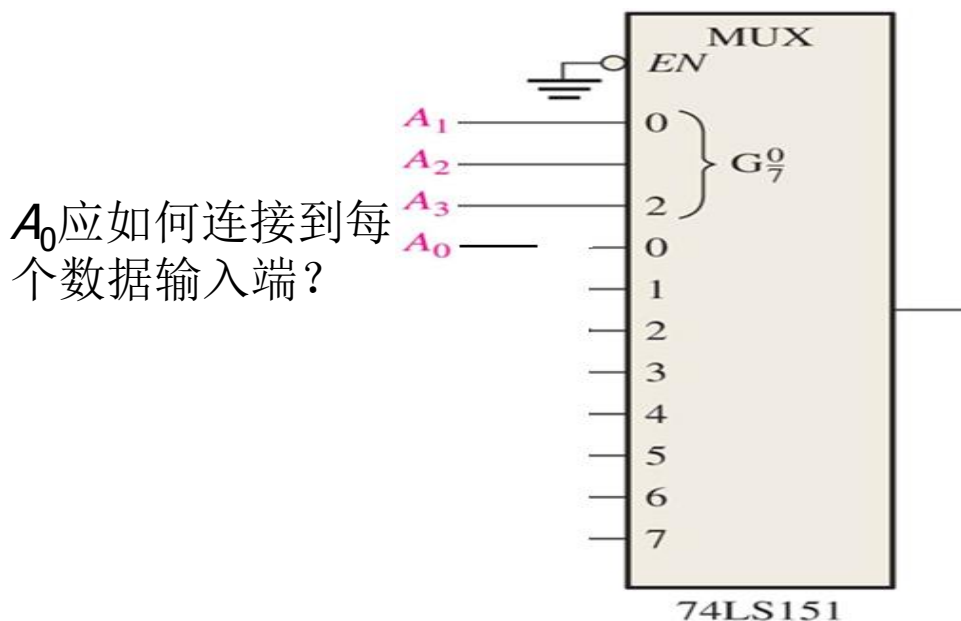


A_2	A_1	A_0	F	D_i
0	0	0	0	D_0
0	0	1	0	D_1
0	1	0	0	D_2
0	1	1	1	D_3
1	0	0	0	D_4
1	0	1	1	D_5
1	1	0	1	D_6
1	1	1	1	D_7

例3 利用一片 74LS151 实现真值表中的逻辑功能。

分析： 74LS151 是 1-of-8 MUX。有3个数据选择端。

如果将 $A_3A_2A_1$ 接到数据选择端， A_0 应接到数据输入端。



decimal digit	inputs $A_3A_2A_1A_0$	output Y
0	0000	0
1	0001	1
2	0010	1
3	0011	0
4	0100	0
5	0101	1
6	0110	1
7	0111	1
8	1000	1
9	1001	0
10	1010	1
11	1011	0
12	1100	1
13	1101	1
14	1110	0
15	1111	1

上页

下页

返回

首先，写出逻辑表达式，再进行适当变换。

$$\begin{aligned}
 Y = & \bar{A}_3 \bar{A}_2 \bar{A}_1 A_0 + \bar{A}_3 \bar{A}_2 A_1 \bar{A}_0 + \bar{A}_3 A_2 \bar{A}_1 A_0 \\
 & + \bar{A}_3 A_2 A_1 \bar{A}_0 + \bar{A}_3 A_2 A_1 A_0 + A_3 \bar{A}_2 \bar{A}_1 \bar{A}_0 \\
 & + A_3 \bar{A}_2 A_1 \bar{A}_0 + A_3 A_2 \bar{A}_1 \bar{A}_0 + A_3 A_2 \bar{A}_1 A_0 \\
 & + A_3 A_2 A_1 A_0
 \end{aligned}$$

$$\begin{aligned}
 Y = & \bar{A}_3 \bar{A}_2 \bar{A}_1 D_0 + \bar{A}_3 \bar{A}_2 A_1 D_1 \\
 & + \bar{A}_3 A_2 \bar{A}_1 D_2 + \bar{A}_3 A_2 A_1 D_3 \\
 & + A_3 \bar{A}_2 \bar{A}_1 D_4 + A_3 \bar{A}_2 A_1 D_5 \\
 & + A_3 A_2 \bar{A}_1 D_6 + A_3 A_2 A_1 D_7
 \end{aligned}$$

decimal digit	inputs A3A2A1A0	output Y
0	0000	0
1	0001	1
2	0010	1
3	0011	0
4	0100	0
5	0101	1
6	0110	1
7	0111	1
8	1000	1
9	1001	0
10	1010	1
11	1011	0
12	1100	1
13	1101	1
14	1110	0
15	1111	1

上页

下页

返回

与MUX的逻辑表达式对比:

$$\begin{aligned}
 Y = & \bar{A}_3 \bar{A}_2 \bar{A}_1 D_0 + \bar{A}_3 \bar{A}_2 A_1 D_1 \\
 & + \bar{A}_3 A_2 \bar{A}_1 D_2 + \bar{A}_3 A_2 A_1 D_3 \\
 & + A_3 \bar{A}_2 \bar{A}_1 D_4 + A_3 \bar{A}_2 A_1 D_5 \\
 & + A_3 A_2 \bar{A}_1 D_6 + A_3 A_2 A_1 D_7
 \end{aligned}$$

$$\begin{aligned}
 Y = & \bar{A}_3 \bar{A}_2 \bar{A}_1 A_0 + \bar{A}_3 \bar{A}_2 A_1 \bar{A}_0 \\
 & + \bar{A}_3 A_2 \bar{A}_1 A_0 + \bar{A}_3 A_2 A_1 \cdot 1 \\
 & + A_3 \bar{A}_2 \bar{A}_1 \bar{A}_0 + A_3 \bar{A}_2 A_1 \bar{A}_0 \\
 & + A_3 A_2 \bar{A}_1 \cdot 1 + A_3 A_2 A_1 A_0
 \end{aligned}$$

$$D_0 = A_0$$

$$D_1 = \bar{A}_0$$

$$D_2 = A_0$$

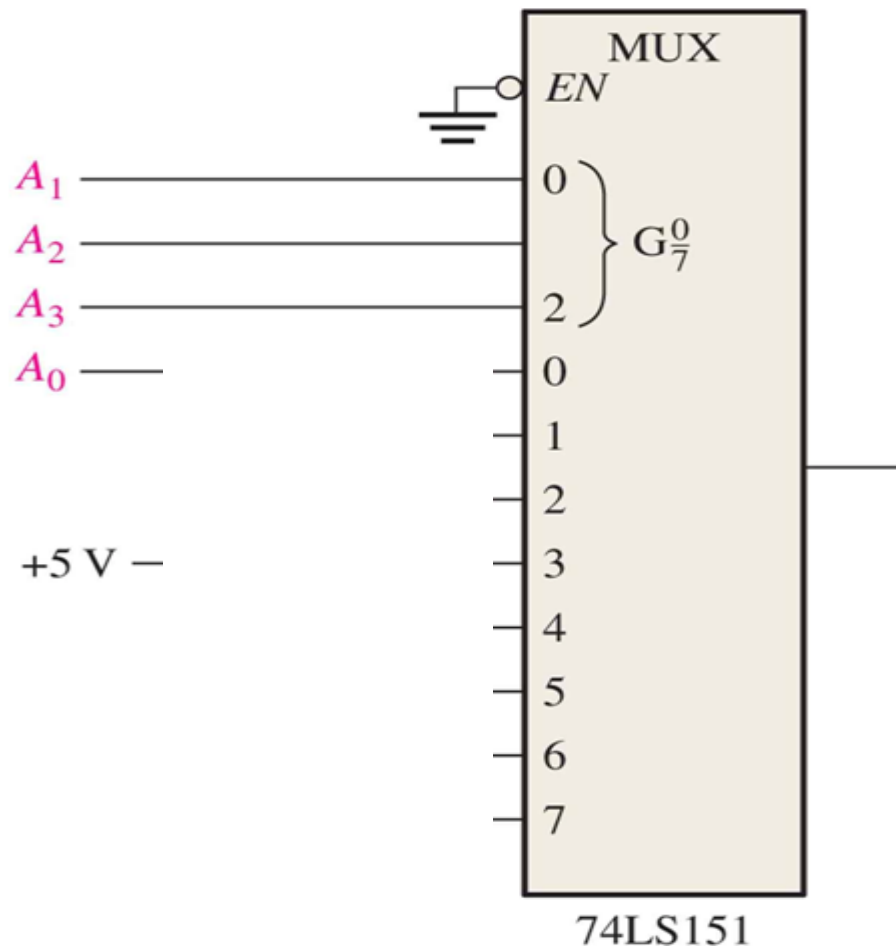
$$D_3 = 1$$

$$D_4 = \bar{A}_0$$

$$D_5 = \bar{A}_0$$

$$D_6 = 1$$

$$D_7 = A_0$$

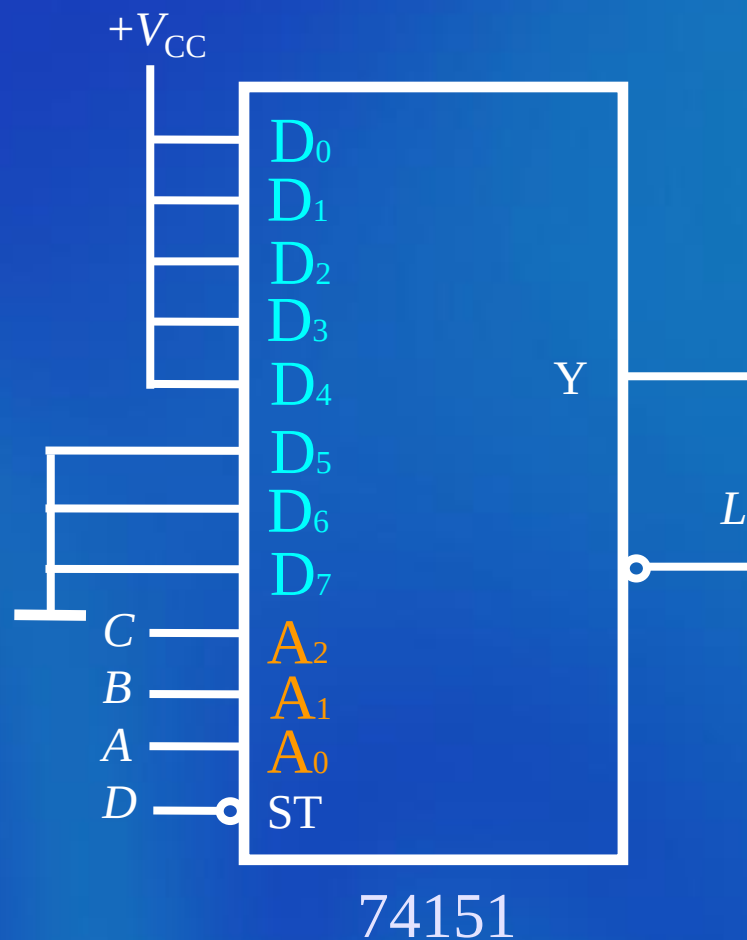


练习 右图中MUX的输入信号
 $DCBA$ 是BCD码，试分析电路
 实现的逻辑功能。

[解] Y 在控制无效时是什么状态？

当 $D=0$ 时， CBA 为
 $000\sim100$ 时， $L=0$ ； CBA 为
 $101\sim111$ 时， $L=1$ ；
 当 $D=1$ 时， $L=1$ 。

实现了四舍五入的逻辑功能。



小结:

- 由于任何组合逻辑函数都可写为最小项的和式，故可以用MUX来实现单输出的逻辑函数。
- 如果逻辑函数的变量数与选用MUX的地址数相同，实现最简单；
- 如果逻辑函数的变量数大于MUX的地址数，可采取扩展的办法增加地址数、将部分变量接到数据输入或控制输入端等办法来解决。

多路分配器(DMUX)

多路分配器与数据选择器相反，它能够把一条通道上的数字信息分时地按 n 位地址分送到 2^n 个数据输出端上。完成这一功能的MSI芯片称为多路数据分配器(Demultiplexer, 简称DMUX)。



多路分配器的示意图

多路分配器可以直接用译码器来实现，或看做是译码器的一种应用。

3-8译码器74LS138构成的1-8DMUX

输入信号D接至 ST_A 端；

将 ST_B 、 ST_C 端接地；

将数据选择ABC作为地址输入端 $A_2 A_1 A_0$ 。

If $CBA=000$,

When $D=0$, the chip is disabled, $\overline{Y}_0=1$;

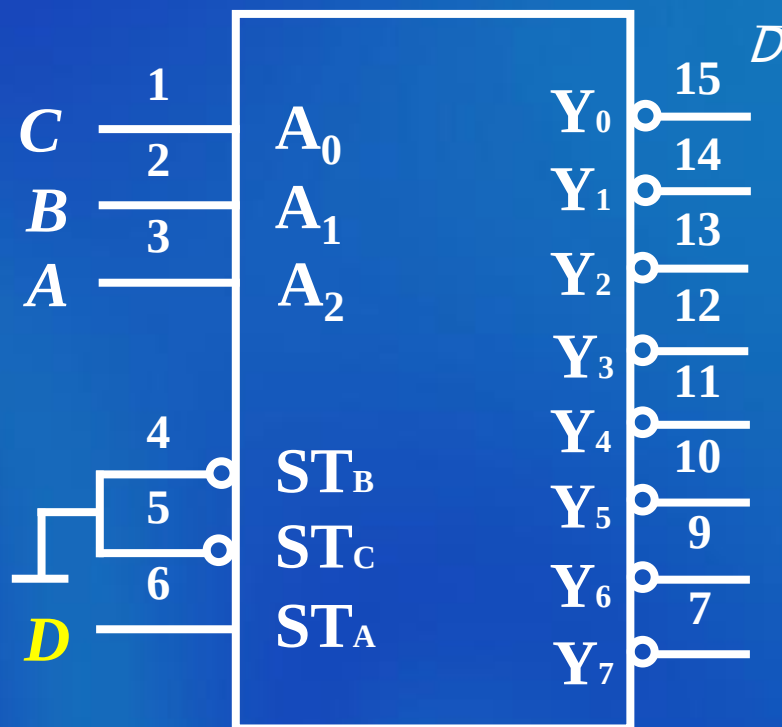
When $D=1$, the chip is enabled, $\overline{Y}_0=0$;

Therefore,
 $\overline{Y}_0 = \overline{D} \overline{C} \overline{B} \overline{A}$

D is distributed to the output Y_0 .

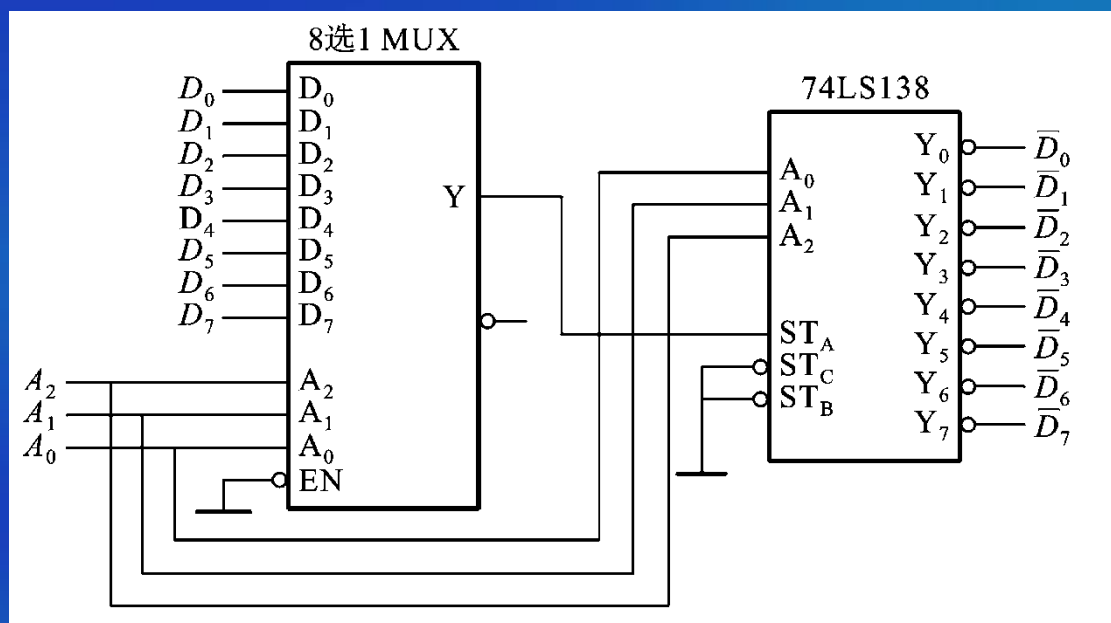
输入数据被反相分配到各输出端。

输入信号D接至 ST_B 或 ST_C 端实现原码分配。



多路选择器和多路分配器配合使用，可以实现在一条数据线上分时传送多路数据的功能。

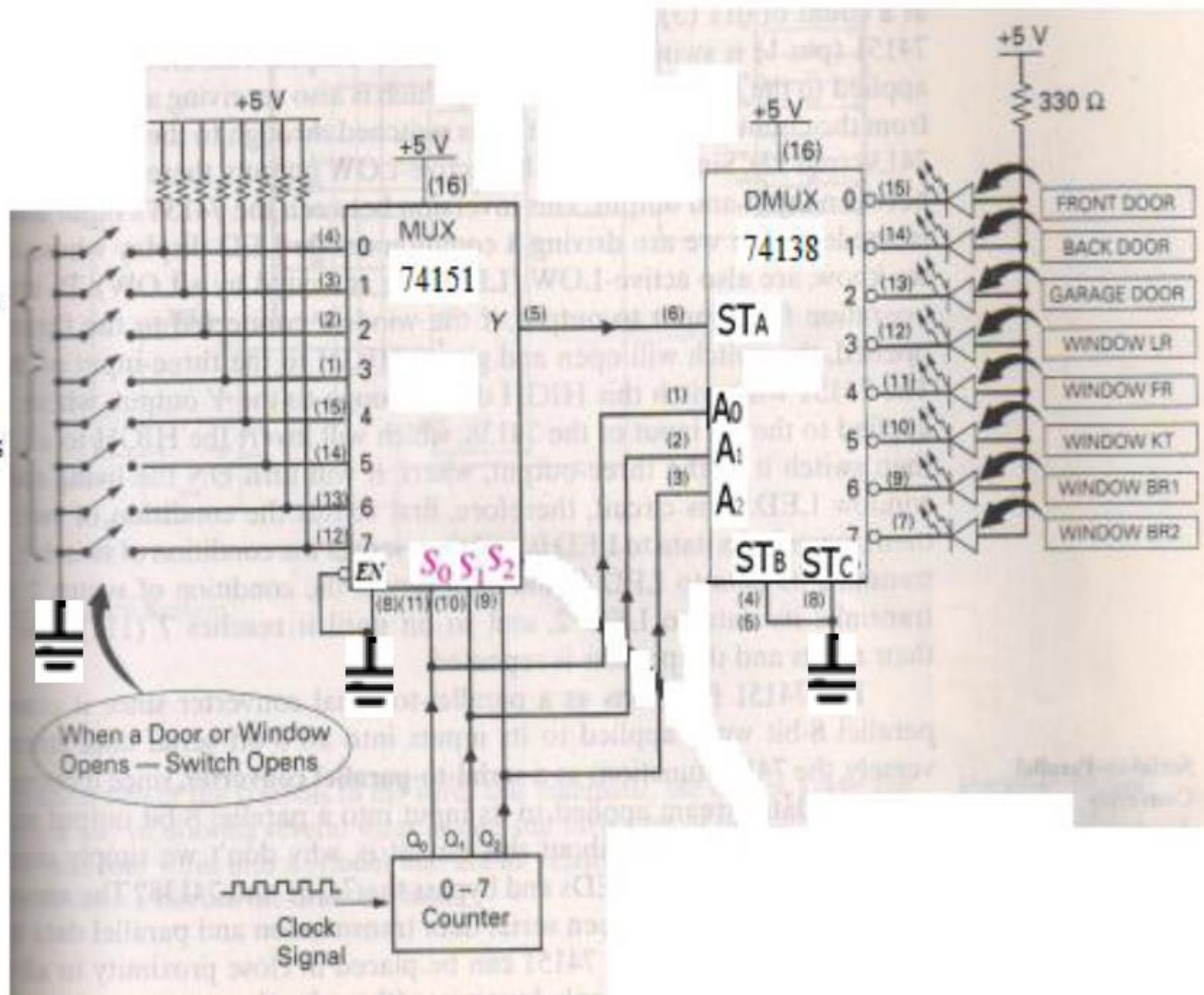
用8选1数据选择器(74151)和八路数据分配器(用74138)实现此目的的逻辑图。



发送端由MUX将各路数据分时送到公共传输线上，接收端再由分配器将公共传输线上的数据适时分配到相应的输出端，而两者的地址输入信号 $A_2 A_1 A_0$ 同步地、顺序地轮流选通。

Door switches

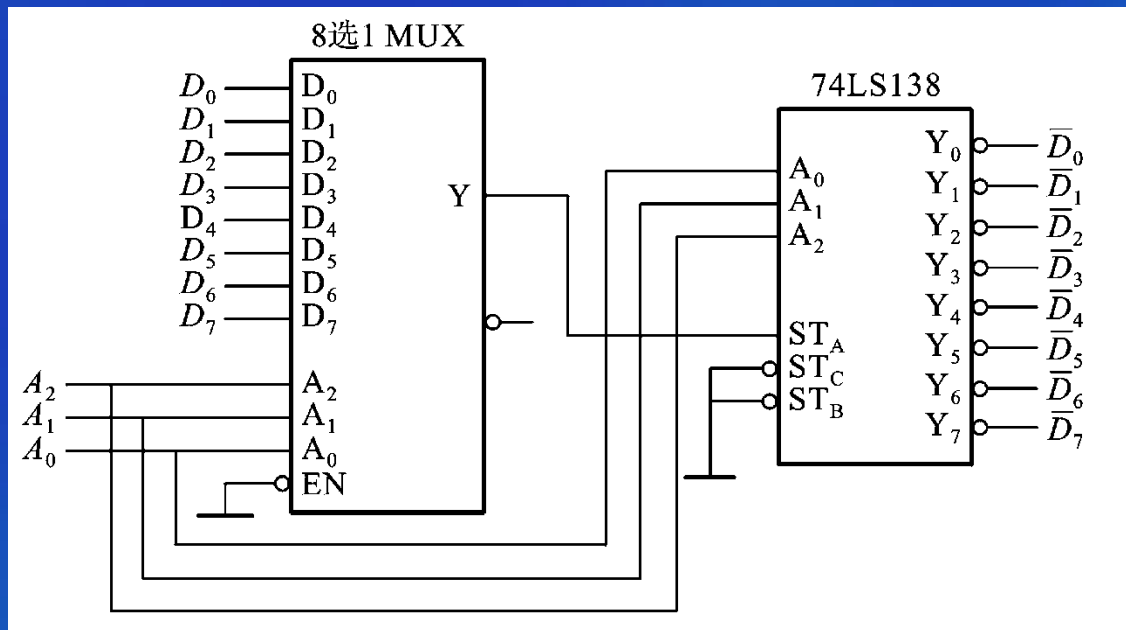
Window switches



上页

下页

返回



在一条数据线上分时传送多路数据的优点：

可大大减少系统中的连接线，尤其是远距离传送数据时，可以节省导线，降低工程造价。

作业

自练题:

- 7.1
- 7.4
- 7.5

作业题:

- 7.6
- 7.7
- 7.8